

Master's Thesis (Academic Year 2026)

Cost Estimation for Distributed Fault-Tolerant Graph State Preparation

Keio University

Graduate School of Media and Governance

Gordon Cui

Abstract

Quantum computing promises dramatic speedups over classical machines for a range of important problems, yet noise and hardware limitations cause error rates high enough to make practical computation infeasible. Building fault-tolerant quantum computers to overcome this has been a major research goal for decades. One route to fault-tolerant computation is measurement-based quantum computing, which consumes fault-tolerant graph states as its computational resources. Producing such a state takes many costly logical operations that can dominate the whole computation and place heavy demands on hardware, especially for the large graphs that real algorithms need.

Prior work scheduled this preparation on a single monolithic processor [1], but the qubit-count limits of one chip make that unrealistic at scale. Distributed quantum computing offers a scalable alternative, with many Quantum Processing Units (QPUs), each holding one logical qubit and linked by photonic entanglement, cooperating to build the state. This flexibility comes at a price, because the cost now depends on several interacting choices, including the type of entangling operation, the number of ancilla QPUs, and the limited connectivity of the optical switches. At present there is no way to predict how these factors combine, so one cannot tell which design is efficient or even practical.

To close this gap we present **DQS-Sim**, a simulator that estimates the cost of distributed graph state preparation under realistic hardware constraints. It implements five scheduling strategies, from idealized all-to-all connectivity to switch-limited operation, and reports the time cost in Tocks, the peak ancilla demand, and the total physical qubit count for any target graph. Built on Litinski’s Game of Surface Codes and checked against known theoretical bounds, DQS-Sim turns these qualitative trade-offs into concrete numbers, giving hardware and protocol designers a practical tool for planning resources. Its central finding is that neither entangling operation dominates: joint measurements prepare faster on the common graph families, but pairwise gates overtake them, by an unbounded factor, on graphs whose measurement conflicts outgrow their degree, and on large two-dimensional graphs whose routing congests a linear substrate, yielding a measured rule for choosing the operation.

Keywords: distributed quantum computing, graph states, measurement-based quan-

tum computing, surface codes, lattice surgery, resource estimation, fault-tolerant quantum computation, edge colouring, maximum independent set.

Contents

Abstract	1
1 Introduction	11
1.1 Background	11
1.2 Research Contribution	12
1.3 Thesis Structure	13
2 Preliminaries	14
2.1 The Quantum Bit	14
2.1.1 The Bloch Sphere	15
2.1.2 Multiple Qubits	16
2.2 Operations on Quantum States	16
2.2.1 Quantum Gates	16
2.2.2 Single-Qubit Gates	16
2.2.3 Two-Qubit Gates	17
2.2.4 Measurement	17
2.3 Entanglement	18
2.4 Graph States	19
2.4.1 Graph Properties	19
2.4.2 Cluster States	20
2.5 Stabilizer Formalism	21
2.5.1 Stabilizers and Stabilizer States	22
2.5.2 Graph States as Stabilizer States	22
2.5.3 Measurements and Syndrome Extraction	22
2.6 Quantum Error Correction	23
2.6.1 Errors	23
2.6.2 No-Cloning Theorem	23
2.6.3 The $[[n,k,d]]$ Stabilizer Code	24
2.6.4 The Surface Code	24
2.7 Patch-Based Surface Code	25
2.7.1 Lattice Surgery Operations	26

2.7.2	Litinski's Game of Surface Codes (GoSC)	26
2.8	Distributed Quantum Computing	30
2.8.1	Architecture	30
2.8.2	Entangling Operations Between QPUs	31
2.8.3	Model Assumptions	32
2.8.4	Switch Connectivity	33
2.9	Chapter Summary	33
3	Problem Statement and Motivation	34
3.1	Related Work	34
3.1.1	The Substrate Scheduler	35
3.1.2	Resource Estimation Frameworks	35
3.2	Formal Problem Statement	36
3.2.1	Notation	37
3.3	The Three Cost Dimensions	37
3.3.1	Dimension 1: Operation Type, MPP versus CZ	37
3.3.2	Dimension 2: Ancilla QPU Pool Size	38
3.3.3	Dimension 3: Switch Connectivity	38
3.4	Research Questions	40
3.5	Scope and Assumptions	40
3.6	Chapter Summary	41
4	Method: DQS-Sim	42
4.1	System Architecture	42
4.1.1	Module Overview	42
4.1.2	Data Flow	42
4.1.3	Command-Line Interface	43
4.2	Case 1: MPP Scheduling, Unlimited Ancillas	43
4.2.1	Algorithm	43
4.2.2	Resource Metrics	45
4.3	Case 2: CZ Scheduling, Unlimited Ancillas	46
4.3.1	Algorithm	46
4.3.2	Resource Metrics	46
4.4	Case 3: CZ Scheduling, Fixed Ancilla Pool	47
4.4.1	Algorithm	47
4.4.2	Resource Metrics	48
4.5	Case 4: MPP Scheduling, Limited Switch Connectivity	48
4.5.1	Switch Feasibility Constraint	49

4.5.2	Algorithm	49
4.6	Case 5: CZ Scheduling, Limited Switch Connectivity	50
4.6.1	Switch Throughput	50
4.6.2	Algorithm	50
4.6.3	Resource Metrics	50
4.7	Physical Resource Calculation	51
4.7.1	Physical Qubits per QPU	51
4.7.2	Total Physical Qubit Count	51
4.7.3	Space-Time Cost	51
4.8	Implementation Details	52
4.8.1	Graph Construction	52
4.8.2	MIS Solver	52
4.8.3	Edge Colouring	52
4.8.4	Verification	53
4.9	Chapter Summary	53
5	Verification and Theoretical Bounds	54
5.1	Case 1: MPP Schedule Bounds	54
5.1.1	Lower Bound via Clique Number	54
5.1.2	Upper Bound via Greedy Colouring	54
5.1.3	Verification Implementation	55
5.2	Case 2: Vizing's Theorem	55
5.2.1	Theoretical Statement	55
5.2.2	Vizing Class	55
5.2.3	Verification Implementation	56
5.3	Case 3: Ancilla-Constrained Monotonicity	56
5.3.1	Monotonicity in k_a	56
5.3.2	Verification Check	57
5.4	Switch Constraint Verification (Cases 4 and 5)	57
5.4.1	Feasibility Condition for MPP (Case 4)	57
5.4.2	Throughput Bound for CZ (Case 5)	57
5.4.3	Consistency Checks	57
5.5	Bound Summary	58
5.6	Chapter Summary	58
6	Results	59
6.1	Experimental Setup	59
6.2	Worked Example: Cycle Graph C_6	60

6.2.1	Cases 1–3: Operation Type and Ancilla Pool	60
6.2.2	Cases 4–5: Switch Connectivity	62
6.3	Worked Example: Complete Graph K_5	65
6.3.1	Cases 1–3: Operation Type and Ancilla Pool	65
6.3.2	Cases 4–5: Switch Connectivity	65
6.3.3	Summary of the Worked Examples	68
6.4	MPP versus CZ	68
6.4.1	Survey at Fixed Size	68
6.4.2	The Conflict Flip	70
6.4.3	The Congestion Flip	72
6.4.4	Which Operation Wins	72
6.5	Scaling with Graph Size	74
6.6	Hardware Sizing: Ancillas, Ports, and Switch Count	75
6.6.1	Scaling the Switch Count	76
6.7	Verification and Cross-Validation	76
6.8	Discussion	78
6.9	Chapter Summary	79
7	Conclusion	80
7.1	Summary of Contributions	80
7.2	Limitations and Future Work	81
7.3	Broader Impact	82

List of Figures

2.1	The Bloch sphere. Every pure qubit state maps to a unique point on the unit sphere, fixed by two angles, θ giving the tilt away from the north pole and ϕ giving the rotation around the equator.	15
2.2	The single-qubit gates used most often in this thesis, each shown as a circuit symbol together with its matrix. The three Pauli gates X , Y , and Z flip the bit value, the phase, or both, and the Hadamard gate H creates an equal superposition.	17
2.3	The two controlled two-qubit gates used in this thesis, each with its matrix. (a) The CZ gate adds a phase of -1 only when both qubits are in the state $ 1\rangle$, and is drawn symmetrically because the two qubits play the same role. (b) The CNOT gate flips the target qubit when the control qubit is in the state $ 1\rangle$	18
2.4	A four-qubit graph state whose graph is a ring. Each circle is a qubit, and each edge marks an entangled pair, here $q_1 - q_2$, $q_2 - q_3$, $q_3 - q_4$, and $q_4 - q_1$	19
2.5	A maximum independent set on the six-vertex cycle C_6 . The green vertices form an independent set of size $\alpha = 3$, no two of which are joined by an edge, while the remaining red vertices cover all of the edges.	21
2.6	A two-dimensional cluster state. Each circle is a qubit and each edge marks the entanglement between a pair of neighbouring qubits.	21
2.7	Surface code patch for code distance $d = 5$	25
2.8	Tile-based surface code representation. (a) Two orientations of a one-qubit patch, where solid edges denote $\bar{Z}$ boundaries and dashed edges denote $\bar{X}$ boundaries. (b) Data patches and an ancilla bus on a tile grid. Each tile represents about $2d^2$ physical qubits.	27
2.9	An MPP measurement of $\bar{Z}_0 \otimes \bar{Z}_1$. An ancilla patch is initialised, merged with both data patches, and then measured. The entire operation costs 1 Tock regardless of the number of target qubits.	28

2.10	CZ gate realised by lattice surgery in two Tocks. In Tock 1 the data patches q_A and q_B merge with an ancilla through an $\bar{X}\bar{X}$ measurement, and in Tock 2 a $\bar{Z}\bar{Z}$ measurement completes the gate and frees the ancilla. Each pairwise CZ gate costs two Tocks.	29
2.11	MIM link between two QPUs	31
2.12	The distributed architecture	31
3.1	Ancilla trade-off curve for Case 3	39
3.2	Two $k = 4$ port switches, each allowing $\lfloor 4/2 \rfloor = 2$ simultaneous CZ pairs. In total, $S\lfloor k/2 \rfloor = 4$ CZ gates can execute per Tock.	39
4.1	DQS-Sim data flow. A target graph and hardware parameters are fed to all five schedulers; outputs are aggregated into a ResourceReport and rendered by the Visualizer.	43
4.2	Ancilla splitting in Case 3	48
6.1	Cycle graph C_6 , with the maximum independent set in green and the vertex cover in red.	60
6.2	Substrate occupancy for C_6 under Case 1 (MPP), one row of strips per Tock. The measured qubit is dark blue, its neighbours light blue, the ancilla patch grey, and hatched cells are blocked positions.	61
6.3	Schedule heatmaps for C_6 under Cases 1 to 3. Blues mark the MPP measurements (Case 1) and oranges the two steps of each CZ gate (Cases 2 and 3).	62
6.4	The C_6 schedules drawn on the graph. Case 1 (MPP) colours each measured vertex and its edges by measurement Tock, with the independent-set vertices in green. Cases 2 and 3 (CZ) colour each edge by its round or batch.	63
6.5	Port wiring for C_6 on one four-port switch. Top row shows Case 4 (MPP) and bottom row Case 5 (CZ).	64
6.6	Schedule heatmaps for C_6 on one four-port switch, Case 4 (MPP) left and Case 5 (CZ) right.	64
6.7	Switch port utilisation for C_6 on one four-port switch, Case 4 (MPP) left and Case 5 (CZ) right.	64
6.8	Complete graph K_5 , with the single-vertex maximum independent set in green and the vertex cover in red.	65
6.9	Schedule heatmaps for K_5 , Case 1 (MPP) on top and the CZ schedule below, which Cases 2, 3, and 5 share on this graph.	66

6.10	The K_5 schedules drawn on the graph. Case 1 (MPP) colours each measured vertex and its edges by measurement Tock, with the single independent-set vertex in green. Cases 2 and 3 (CZ) colour each edge by its round or batch.	67
6.11	Switch port utilisation for K_5 on one four-port switch. Only Case 5 (CZ) appears, since Case 4 (MPP) is infeasible here.	67
6.12	Port wiring for K_5 on one four-port switch. Left shows the infeasible Case 4 (MPP) measurement and right one Case 5 (CZ) round.	67
6.13	Schedule depth by graph family and case ($k_a = 2$, one four-port switch). Bars are labelled with the Tock count and $\times$ marks infeasible Case-4 instances.	69
6.14	The Heawood graph, the smallest incidence graph in the family, $PG(2,2)$. Blue vertices are its seven points and gold its seven lines, with an edge whenever a point lies on a line.	71
6.15	MPP versus CZ. (a) Schedule depth on the incidence family. (b) Schedule depth on square grids, with the flat conflict floor dotted. (c) Every tested graph placed by conflict ratio against measured depth ratio. The gold upper half holds the CZ wins, the blue lower half the MPP wins, and the red triangles are the large grids.	73
6.16	Measured schedule depth versus graph size for Case 1 (MPP, blue) and Case 2 (CZ, orange) on four graph families.	74
6.17	Measured hardware trade-off curves for C_{12} (blue) and K_8 (orange). (a) Case-3 schedule depth versus ancilla pool size k_a ; dotted lines mark each graph's unlimited-ancilla floor $T_2 = 2\chi'$. (b) Case-5 depth versus switch port count k ; dashed vertical lines mark the smallest switch on which Case 4 (MPP) becomes feasible, at $k = \Delta + 2$	75
6.18	One Tock of Case 4 (MPP) for C_8 on two four-port switches, one measurement per switch.	77
6.19	One round of Case 5 (CZ) for C_8 on two four-port switches, two gates per switch.	77
6.20	Measured Case-5 (CZ) depth versus switch count at fixed $k = 4$, with each graph's unlimited floor dotted.	77

List of Tables

2.1	Comparison of MPP and CZ operations in the GoSC framework.	29
3.1	Comparison of resource estimation frameworks for fault-tolerant quantum computing.	36
3.2	Notation used throughout the thesis	37
4.1	Resource formulae for the five DQS-Sim cases. Here $\ell = S\lfloor k/2 \rfloor$ is the CZ throughput of the switches, and $g_{\max}$ is the largest number of measurements scheduled in a single Tock in Case 4. Each concurrent measurement uses one ancilla QPU, and measurements are packed across the S switches with each switch hosting measurements whose combined port demand, data qubits plus one ancilla port per measurement, fits within its k ports.	52
5.1	Bounds for each DQS-Sim case as enforced at runtime. Case 4 has no closed-form upper bound, so its schedule depth is reported as measured.	58
6.1	DQS-Sim results for the worked examples at code distance $d = 7$. Cases 4 and 5 use a single $k = 4$ -port switch ($S = 1$). T_c/T_1 is the Tock ratio relative to Case 1.	68
6.2	Schedule depth in Tocks for six graph families under all five cases ($k_a = 2$ ancillas for Case 3; one $k = 4$ -port switch for Cases 4 and 5). $\times$ marks Case-4 instances that are infeasible because a generator needs $\Delta + 2 > 4$ ports.	69
6.3	The conflict flip. $\omega(\mathcal{C})$ is the exact clique number of the generator conflict graph and χ' the greedy chromatic index, so $T_2 = 2\chi'$. Comparing ω against $2\chi'$ identifies the faster operation on all ten graphs.	71
6.4	The congestion flip on square grids. The conflict clique stays flat at $\omega = 4$, yet T_1 grows with the grid side and overtakes the constant $T_2 = 8$ at 6×6	72

Chapter 1

Introduction

1.1 Background

Quantum computing has advanced rapidly over recent decades, growing from a theoretical idea into one of the most actively pursued goals in science and engineering [2]. By exploiting the principles of quantum mechanics, a quantum computer can in principle solve certain problems that are impractical for any classical machine [3, 4]. This promise, often called quantum advantage, has driven rapid progress in both hardware and theory. Yet today’s quantum devices remain small and highly sensitive to noise [5], and this noise corrupts quantum information faster than a large computation can complete. Realising the full potential of quantum computing therefore depends on a transition from these noisy near-term devices to fault-tolerant machines, in which quantum error correction protects the computation as it runs.

Quantum error correction comes at a steep price. Protecting a single logical qubit requires encoding it across many physical qubits, so the number and quality of the available qubits will limit applications for years to come. Making good use of these scarce resources is essential, which places a premium on tools that can estimate and reduce the cost of running fault-tolerant programs.

One particularly flexible route to fault-tolerant computation is measurement-based quantum computing [6]. In this model the computation is carried out by measuring a large entangled state known as a graph state, which serves as the computational resource that an algorithm consumes. Turning a high-level algorithm into a fault-tolerant computation can be viewed as a pipeline, in which the algorithm is first translated into the graph state it requires, that graph state is then prepared on error-corrected hardware, and the hardware finally carries out the measurements that drive the computation. The preparation of the graph state sits at the heart of this pipeline, and because every vertex of the state is a fault-tolerant logical qubit, its preparation can dominate the cost of the entire computation.

Until recently, graph state preparation was studied mainly on a single monolithic

processor, where all logical qubits share one chip [1]. A single chip can hold only a limited number of qubits, so this approach cannot reach the large graph states that practical algorithms demand. Distributed quantum computing removes this ceiling by linking many small Quantum Processing Units (QPUs), each holding one logical qubit, through photonic entanglement. The move to a distributed machine, however, introduces a web of new hardware choices, including the type of entangling operation, the size of the ancilla pool, and the limited connectivity of the optical switches that join the units. How these choices combine to determine the cost of preparing a graph state is not yet understood, and no existing tool can answer it.

This thesis addresses that question directly. We present DQS-Sim, a simulator that estimates the fault-tolerant cost of preparing graph states across a network of distributed processors, making it possible to compare hardware choices and to judge which designs are efficient and practical. The model operates at the logical level throughout: physical-layer mechanisms such as photon loss and entanglement purification are not simulated, and Chapter 2 states the explicit assumptions under which they can be set aside.

1.2 Research Contribution

This thesis presents DQS-Sim, a simulator that estimates the fault-tolerant cost of preparing graph states on a distributed quantum computer. The main contributions are as follows.

- **A distributed cost simulator.** DQS-Sim estimates the cost of graph state preparation across five scheduling scenarios of increasing hardware realism, ranging from idealised all-to-all connectivity to switch-limited operation.
- **Two operation models under realistic limits.** The simulator compares multi-Pauli-product measurement and CZ-gate preparation, and quantifies how their cost changes as the ancilla pool and the switch connectivity are restricted.
- **Unified resource reporting.** For any target graph, DQS-Sim reports the schedule depth in Ticks, the peak ancilla QPU demand, and the total physical qubit count in a single view, using Litinski’s Game of Surface Codes as the underlying cost model.
- **Verified schedulers.** Each scheduling algorithm is checked against established theoretical bounds, giving confidence that the reported costs are sound.

- **Design guidance.** The results turn previously qualitative trade-offs into concrete numbers, giving hardware and protocol designers a practical basis for planning resources.

1.3 Thesis Structure

The remainder of this thesis is organised as follows. Chapter 2 introduces the quantum information and quantum error correction concepts used throughout, including qubits, entanglement, graph states, the surface code, measurement-based quantum computing, and the distributed architecture in which this work is set. Chapter 3 formalises the distributed graph state cost estimation problem, identifies the factors that drive cost, and reviews the related work that this thesis builds on. Chapter 4 describes DQS-Sim and its five scheduling algorithms. Chapter 5 checks the schedulers against theoretical bounds. Chapter 6 presents resource estimates across representative graph families. Chapter 7 summarises the findings and outlines directions for future work.

Chapter 2

Preliminaries

This chapter introduces the basics of quantum information and establishes the theoretical foundations required to understand distributed fault-tolerant quantum computing (DQC) and graph state preparation. We cover the quantum bit, multi-qubit operations, the stabilizer formalism, and the surface code error-correction scheme that supports the fault-tolerant protocols analysed in this thesis.

2.1 The Quantum Bit

A quantum bit, or qubit, is the quantum counterpart of the classical bit and the smallest unit of information in a quantum computer. A classical bit is always in one of two states, 0 or 1. A qubit can instead be in the state $|0\rangle$, in the state $|1\rangle$, or in any superposition of the two at the same time. This ability to hold both values at once has no classical analogue and is one of the main sources of a quantum computer's power.

Quantum states are written using the Dirac notation, also called bra-ket notation. The two basis states $|0\rangle$ and $|1\rangle$ are the direct counterparts of the classical values 0 and 1, and a general single-qubit state is written as a linear combination of them,

$$|\psi\rangle = \alpha |0\rangle + \beta |1\rangle. \quad (2.1)$$

The coefficients α and β are complex numbers called amplitudes, and the basis states $|0\rangle$ and $|1\rangle$ are simply the special cases where one amplitude is zero. The amplitudes are not free, since they must satisfy the normalisation condition $|\alpha|^2 + |\beta|^2 = 1$.

A single-qubit state can also be represented as a column vector. Writing the basis states as

$$|0\rangle = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad |1\rangle = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \quad (2.2)$$

a general qubit state becomes the column vector of its two amplitudes,

$$|\psi\rangle = \alpha |0\rangle + \beta |1\rangle = \begin{bmatrix} \alpha \\ \beta \end{bmatrix}. \quad (2.3)$$

When a qubit is measured in the computational basis, the result is 0 with probability $|\alpha|^2$ and 1 with probability $|\beta|^2$. The measurement also disturbs the qubit, collapsing it irreversibly onto whichever basis state was observed.

2.1.1 The Bloch Sphere

The state of a single qubit is often represented as a point on the surface of a sphere, called the Bloch sphere shown in Figure 2.1. The north pole is the state $|0\rangle$ and the south pole is the state $|1\rangle$, and every other point on the surface is some superposition of the two. A single-qubit state can be placed on the sphere using two angles θ and ϕ ,

$$|\psi\rangle = \cos \frac{\theta}{2} |0\rangle + e^{i\phi} \sin \frac{\theta}{2} |1\rangle, \quad (2.4)$$

where θ sets how far the point tilts away from the north pole and ϕ sets its rotation around the equator.

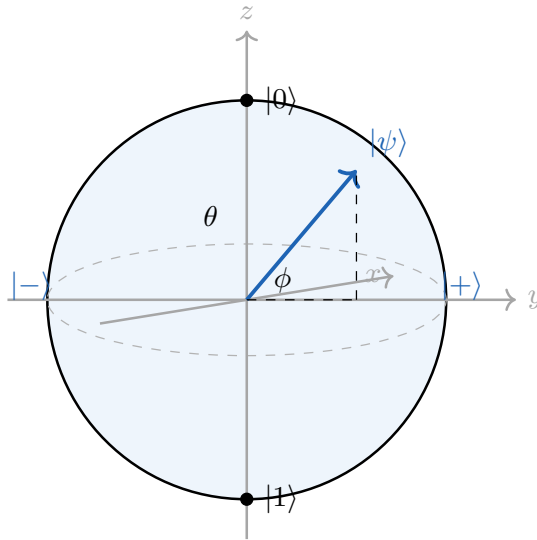


Figure 2.1: The Bloch sphere. Every pure qubit state maps to a unique point on the unit sphere, fixed by two angles, θ giving the tilt away from the north pole and ϕ giving the rotation around the equator.

2.1.2 Multiple Qubits

By combining multiple single qubits, we can obtain multi-qubit quantum states. With n qubits there are 2^n basis states, each one obtained by listing the value of every qubit. For two qubits the four basis states are $|00\rangle$, $|01\rangle$, $|10\rangle$, and $|11\rangle$, and a general two-qubit state is a superposition of all four,

$$|\psi\rangle = \alpha_{00} |00\rangle + \alpha_{01} |01\rangle + \alpha_{10} |10\rangle + \alpha_{11} |11\rangle, \quad (2.5)$$

with the same rule as before that the squared amplitudes add up to one. Because the number of amplitudes doubles with every qubit added, the description of the system grows exponentially with the number of qubits. This exponential growth is what lets a quantum computer hold and process information that is far beyond the reach of any classical machine.

2.2 Operations on Quantum States

There are two basic ways to act on a qubit, by applying gates that transform its state and by measuring it to read out information. This section covers both.

2.2.1 Quantum Gates

Just as a classical computer transforms bits with logic gates, a quantum computer transforms qubits with quantum gates. A quantum gate is a reversible operation, described by a unitary matrix that preserves the length of the state vector. Because the operation is reversible, every quantum gate can be undone by applying its inverse.

Mathematically, the action of a gate on a qubit corresponds to multiplying the qubit's state vector by the gate's matrix. For instance, applying the X gate to $|0\rangle$ yields

$$X |0\rangle = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \end{bmatrix} = |1\rangle, \quad (2.6)$$

recovering the expected bit flip. The composition of several gates corresponds to the product of their matrices, so an entire circuit reduces to a single unitary acting on the input state.

2.2.2 Single-Qubit Gates

The simplest gates act on a single qubit. The three Pauli gates and the Hadamard gate, shown in Figure 2.2, are the ones used most often in this thesis. The Pauli X

gate exchanges $|0\rangle$ and $|1\rangle$, so it plays the role of the classical NOT gate. The Pauli Z gate leaves $|0\rangle$ unchanged and attaches a minus sign to $|1\rangle$, which is called a phase flip. The Pauli Y gate combines a bit flip and a phase flip. The Hadamard gate H turns a definite state into an equal superposition, mapping $|0\rangle$ to $|+\rangle = (|0\rangle + |1\rangle)/\sqrt{2}$ and $|1\rangle$ to $|-\rangle = (|0\rangle - |1\rangle)/\sqrt{2}$, and it is the gate that creates superposition in the first place.

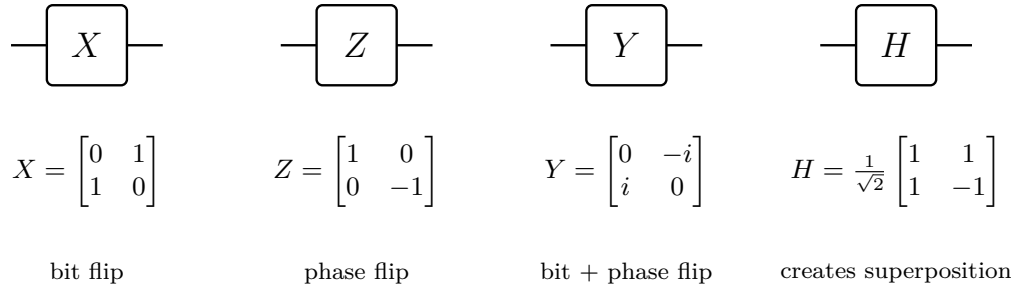


Figure 2.2: The single-qubit gates used most often in this thesis, each shown as a circuit symbol together with its matrix. The three Pauli gates X , Y , and Z flip the bit value, the phase, or both, and the Hadamard gate H creates an equal superposition.

2.2.3 Two-Qubit Gates

To build entanglement we need gates that act on two qubits at once. A controlled gate uses one qubit as a control and applies an operation to a second qubit depending on the value of the control. Two such gates appear in this thesis, both shown in Figure 2.3.

The controlled-NOT (CNOT) gate flips the second qubit exactly when the control qubit is in the state $|1\rangle$, which makes it the quantum version of the classical XOR and a common tool for creating entanglement. The controlled- Z (CZ) gate instead leaves the two qubits unchanged unless both are in the state $|1\rangle$, in which case it multiplies the state by -1 . The CZ gate is the one at the centre of this thesis, and two properties make it convenient to work with. It is symmetric, so it does not matter which qubit is treated as the control, and it is its own inverse, since applying it twice returns the original state.

The CZ gate is one of the basic building blocks used to create graph states, as later sections describe.

2.2.4 Measurement

Measurement is how information is read out of a qubit, and unlike a gate it cannot be undone. Measuring a qubit in the computational basis returns the outcome 0 with probability $|\alpha|^2$ and 1 with probability $|\beta|^2$, and the act of measuring collapses the

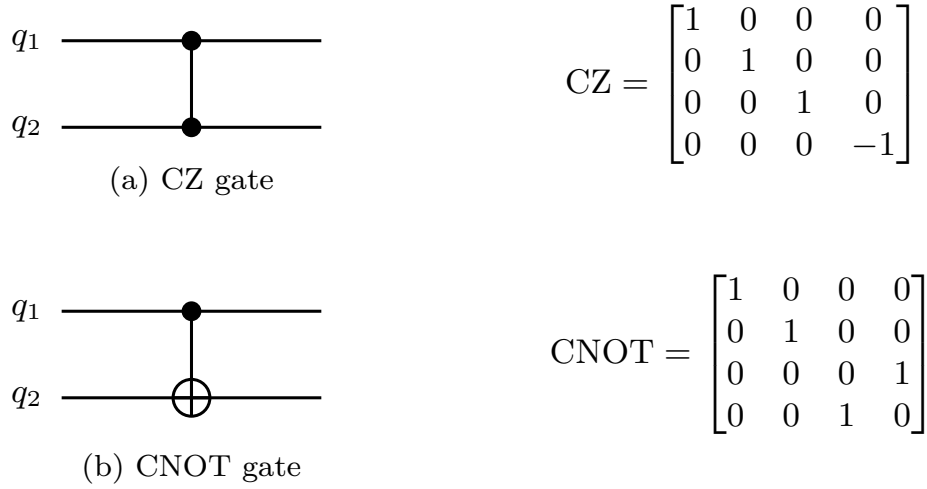


Figure 2.3: The two controlled two-qubit gates used in this thesis, each with its matrix. (a) The CZ gate adds a phase of -1 only when both qubits are in the state $|1\rangle$, and is drawn symmetrically because the two qubits play the same role. (b) The CNOT gate flips the target qubit when the control qubit is in the state $|1\rangle$.

qubit onto the observed value, erasing the superposition that was present before. A qubit need not be measured in the computational basis. It can equally be measured along any axis of the Bloch sphere, for example in the basis formed by the states $|+\rangle$ and $|-\rangle$, or more generally by projecting onto the eigenstates of an observable such as a Pauli operator. Measurements of this standard kind are known as projective, or von Neumann, measurements, and they are the form of measurement that underlies measurement-based quantum computing [6], in which an entire computation is carried out through a carefully chosen sequence of single-qubit measurements on a graph state.

2.3 Entanglement

Entanglement is a uniquely quantum property that separates quantum systems from classical ones. When two or more qubits are entangled, their states are linked so tightly that we cannot describe one qubit on its own, no matter how far apart they are. Measuring one of them immediately reveals something about the others. A state is called separable when it can be split into independent single-qubit states, and entangled when no such split is possible.

The simplest entangled states are the four Bell states,

$$|\Phi^+\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle), \quad |\Phi^-\rangle = \frac{1}{\sqrt{2}}(|00\rangle - |11\rangle), \quad (2.7)$$

$$|\Psi^+\rangle = \frac{1}{\sqrt{2}}(|01\rangle + |10\rangle), \quad |\Psi^-\rangle = \frac{1}{\sqrt{2}}(|01\rangle - |10\rangle). \quad (2.8)$$

Taking the first of these, $|\Phi^+\rangle$, the two qubits are always found to agree, both 0 or both 1, even though neither qubit has a definite value on its own. Bell states are maximally entangled, meaning the qubits are as strongly correlated as quantum mechanics allows. They are the basic resource for tasks such as quantum teleportation, and in this thesis they are the entanglement shared between distributed processors over photonic links.

2.4 Graph States

A graph state [7] is a multi-qubit entangled state that is described by a graph $G = (V, E)$. Each vertex in V is a qubit, and each edge in E marks a pair of qubits that are entangled with each other. Describing a quantum state with a graph in this way is far more compact than listing all of its amplitudes, and it is what makes graph states the main computational resource of measurement-based quantum computing.

As a concrete example, Figure 2.4 shows a graph state on four qubits whose graph is a ring. The four edges record that qubit 1 shares entanglement with qubits 2 and 4, qubit 2 with 1 and 3, and so on around the cycle. The graph itself is simple, yet the state it represents is a genuinely entangled four-qubit state.

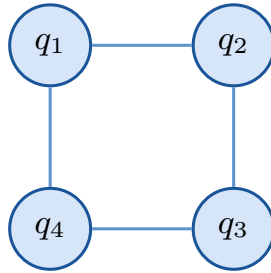


Figure 2.4: A four-qubit graph state whose graph is a ring. Each circle is a qubit, and each edge marks an entangled pair, here $q_1 - q_2$, $q_2 - q_3$, $q_3 - q_4$, and $q_4 - q_1$.

2.4.1 Graph Properties

Several properties of the underlying graph are used repeatedly in later chapters.

Neighbours and degree. The neighbourhood of a vertex a , denoted $N(a)$, is the set of vertices adjacent to it,

$$N(a) = \{ b \in V \mid \{a, b\} \in E \}. \quad (2.9)$$

The *degree* of a vertex is the number of edges touching it, and $\Delta(G)$ is the largest degree in the graph. A vertex of degree five is a qubit that must take part in five entangling operations, so degree measures how busy a qubit is. The graph as a whole can be recorded as an adjacency matrix Γ , whose entry Γ_{ab} equals 1 when a and b are adjacent and 0 otherwise.

Independent sets and vertex covers. A set of vertices is *independent* if no two of its members are adjacent, and the largest one is the maximum independent set (MIS), of size $\alpha(G)$. The vertices left outside form a *vertex cover*, a set that touches every edge of the graph. Figure 2.5 shows both on the six-vertex cycle, the green MIS and the red cover. In Chapter 4 the independent-set qubits come for free and only the cover needs measuring.

Edge colouring. An *edge colouring* gives every edge a colour so that no two edges sharing a vertex receive the same one. Edges of one colour never touch, so their gates can all run at the same time, and the fewest colours needed, the *chromatic index* $\chi'(G)$, is the depth of the best round structure for two-qubit gates. On the six-vertex cycle two alternating colours suffice, so $\chi'(C_6) = 2$.

Cliques and conflict graphs. A *clique* is a set of vertices all connected to each other. Cliques enter the scheduling story through a derived graph used in later chapters, the *conflict graph*, which has one vertex per operation and an edge wherever two operations need the same qubit and so cannot run at the same time. A clique there is a set of operations that all block one another, so the size of the largest clique, written ω , is a floor on the length of any schedule.

2.4.2 Cluster States

A particularly important family of graph states is the cluster states [8], whose graphs are regular lattices, most commonly two-dimensional, as shown in Figure 2.6. Cluster states were the first states proven to be a universal resource for measurement-based quantum computing [6, 9]. A general graph state relaxes this regular structure and allows an arbitrary graph, which is what enables graph states to encode the structure

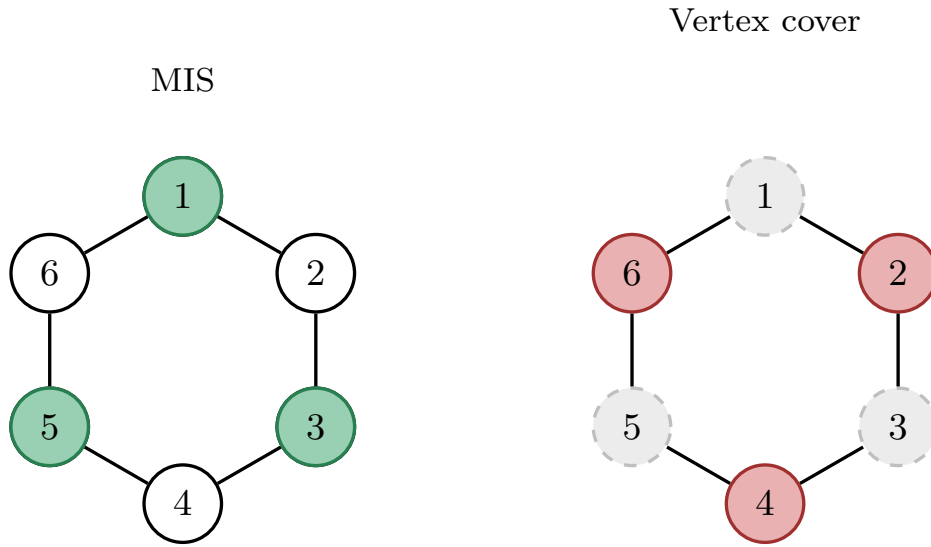


Figure 2.5: A maximum independent set on the six-vertex cycle C_6 . The green vertices form an independent set of size $\alpha = 3$, no two of which are joined by an edge, while the remaining red vertices cover all of the edges.

of an arbitrary algorithm.

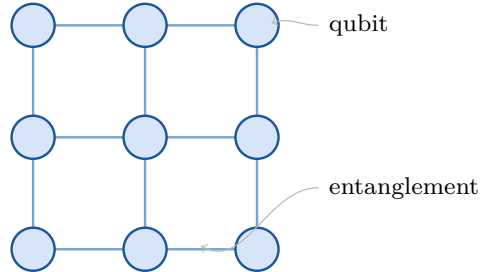


Figure 2.6: A two-dimensional cluster state. Each circle is a qubit and each edge marks the entanglement between a pair of neighbouring qubits.

The procedure for preparing a graph state on physical hardware, by entangling qubits in accordance with the edges of G , is the central concern of this thesis and is developed in the chapters that follow.

2.5 Stabilizer Formalism

A general state of n qubits is described by its 2^n amplitudes, a list that grows too large to handle as n increases. The stabilizer formalism [10] provides a far more compact description for an important family of states, specifying each state not by its amplitudes

but by the operators that leave it unchanged. This family includes both the graph states of the previous section and the surface code introduced later.

2.5.1 Stabilizers and Stabilizer States

An operator S is said to stabilize a state $|\psi\rangle$ when the state is a +1 eigenstate of S ,

$$S|\psi\rangle = |\psi\rangle. \quad (2.10)$$

Equivalently, applying S leaves the state unchanged. For instance, the state $|+\rangle$ is stabilized by the Pauli operator X , since $X|+\rangle = |+\rangle$, and the state $|0\rangle$ is stabilized by Z . The operators used here are Pauli operators, that is, tensor products of the single-qubit operators I , X , Y , and Z .

A stabilizer state on n qubits is a state that is fixed in this way by a whole group of commuting Pauli operators, known as its stabilizer group. Listing the entire group is unnecessary, because it follows from a set of n independent generators. Describing a state with n Pauli operators rather than 2^n amplitudes is a substantial saving, and it is what makes stabilizer states efficient to store and to simulate on a classical computer. Stabilizer states are precisely the states that can be reached from $|0\rangle^{\otimes n}$ using the Clifford gates, namely H , the phase gate S , and CNOT or the equivalent CZ.

2.5.2 Graph States as Stabilizer States

Graph states form an important subclass of stabilizer states, and their generators take a particularly simple form that mirrors the graph directly. Each vertex i contributes one generator,

$$g_i = X_i \bigotimes_{j \in N(i)} Z_j, \quad (2.11)$$

which applies X to qubit i and Z to every qubit in its neighbourhood $N(i)$. The graph state $|G\rangle$ is the unique state satisfying $g_i |G\rangle = |G\rangle$ for all i , so the connectivity of the graph is written directly into the stabilizers.

Example 2.1 (Four-qubit ring). For the ring of Figure 2.4, the four generators are $g_1 = X_1 Z_2 Z_4$, $g_2 = X_2 Z_1 Z_3$, $g_3 = X_3 Z_2 Z_4$, and $g_4 = X_4 Z_1 Z_3$.

2.5.3 Measurements and Syndrome Extraction

Measuring a Pauli operator on a stabilizer state either commutes or anti-commutes with each generator. When it commutes with every generator the outcome is fixed in

advance, whereas when it anti-commutes with some generator the outcome is random and that generator is replaced by the measured operator.

In error correction, a fixed set of stabilizer generators is measured over and over. The pattern of their outcomes is called the error syndrome, and it signals which errors have occurred, and roughly where, without revealing the logical information that the state encodes. The errors can then be corrected while the computation continues.

2.6 Quantum Error Correction

Physical qubits are fragile, and any real quantum computer must cope with the errors that noise introduces. Quantum error correction [11, 12] is the set of techniques that protect quantum information against these errors, allowing a noisy device to behave as a reliable, fault-tolerant machine. This section introduces the errors themselves, the no-cloning theorem that rules out the simplest classical defence against them, and the error-correcting codes, in particular the surface code, that this thesis relies on.

2.6.1 Errors

A qubit is never perfectly isolated. Interactions with its environment disturb its state, a process known as decoherence, and these disturbances are what we refer to as errors. Any single-qubit error can be expressed in terms of the Pauli operators, so it is enough to correct two basic types. A bit-flip error is an unwanted application of X , which exchanges $|0\rangle$ and $|1\rangle$, while a phase-flip error is an unwanted application of Z , which flips the sign of $|1\rangle$. A Y error is simply a combination of the two. The chance of an error is summarised by a physical error rate p per operation. Error correction becomes worthwhile once p falls below a threshold value, beyond which adding more physical qubits steadily drives the logical error rate down.

2.6.2 No-Cloning Theorem

A natural way to protect classical information is to copy it, yet quantum information cannot be copied. The no-cloning theorem [13] states that no operation can produce an identical, independent copy of an arbitrary unknown quantum state.

There are two natural ways one might try to copy a qubit, and both fail. One is to measure the qubit and reproduce the result. A measurement, however, collapses the state onto a single basis outcome and destroys any superposition before it can be learned, so the original is lost rather than copied. The other is to apply a quantum operation that copies the state directly. Because every quantum operation is linear

and unitary, it can be shown that applying such a cloning operator to a superposition produces an entangled state of the two qubits rather than two identical copies. No unitary operation can therefore copy an arbitrary unknown state.

This result shapes the rest of the thesis in two ways. First, quantum error correction cannot duplicate a qubit and take a majority vote, as classical codes do. It must instead spread a single logical qubit across many entangled physical qubits, so that errors can be detected and corrected without ever reading or copying the stored state. Second, in a distributed machine a quantum state cannot simply be copied and sent from one processor to another. Communication must instead consume shared entanglement, in the form of the Bell pairs introduced earlier, which is precisely the mechanism that links the processors in the architecture studied in this thesis.

2.6.3 The $[[n,k,d]]$ Stabilizer Code

A $[[n, k, d]]$ quantum error-correcting code encodes k logical qubits into n physical qubits with code *distance* d : the minimum weight of an operator that maps one logical codeword to another. A distance- d code can detect and correct any error acting on fewer than $d/2$ physical qubits.

The logical error rate scales as $p_L \approx (p/p_{\text{th}})^{\lfloor d/2 \rfloor}$, where p_{th} is the code's *threshold*. If $p < p_{\text{th}}$, increasing d suppresses errors exponentially.

2.6.4 The Surface Code

Code Definition

The *surface code* [14–17] is a topological stabilizer code defined on a 2-D square lattice. A distance- d patch contains d^2 data qubits and $d^2 - 1$ ancilla (syndrome) qubits arranged in a checkerboard pattern of X - and Z -type stabilizers, as shown in Figure 2.7. The total qubit count per logical qubit is

$$n_{\text{phys}}(d) = 2d^2 - 1. \quad (2.12)$$

This thesis uses $d = 7$ throughout, the simulator's default and an illustrative mid-range value, giving $n_{\text{phys}} = 97$ physical qubits per logical qubit (QPU). The exact choice is not critical, since the distance only sets this per-QPU constant (Chapter 4 discusses the choice).

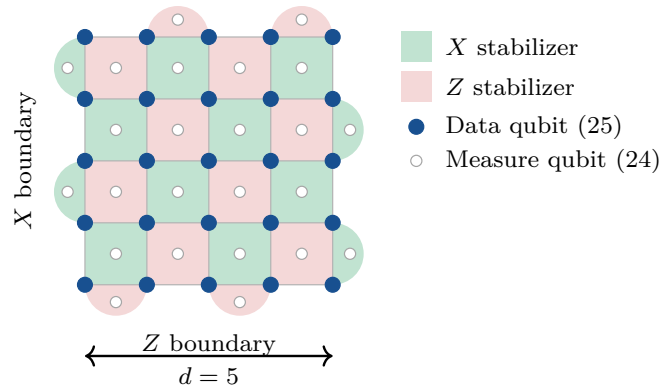


Figure 2.7: Surface code patch for code distance $d = 5$. Data qubits (filled circles) sit at the lattice vertices and measure qubits (open circles) sit inside the checks. X - and Z -type stabilizers alternate in a checkerboard, and the weight-2 boundary checks appear as semicircles. A distance- d patch uses $2d^2 - 1$ physical qubits.

Error Threshold

The surface code has one of the highest known thresholds: $p_{\text{th}} \approx 1\%$ under circuit-level depolarising noise [17]. This makes it the leading candidate for fault-tolerant quantum computation on near-term hardware.

Logical Operators

A distance- d surface code patch supports one logical qubit. The logical $\bar{Z}$ operator is a chain of d Pauli Z operators connecting the patch's two Z boundaries, and the logical $\bar{X}$ is a chain of d Pauli X operators connecting its two X boundaries, the boundary labels of Figure 2.7. Any physical error chain shorter than $d/2$ is correctable.

2.7 Patch-Based Surface Code

There are several types of surface code, including defect-based, twist-based, and patch-based encodings. This thesis uses the patch-based surface code framework introduced in Litinski's Game of Surface Codes. In this framework the quantum hardware is viewed as a board made up of tiles, and each logical qubit is stored as a patch that occupies one or more of these tiles. Logical operations are then carried out by lattice surgery, which merges and splits the boundaries of neighbouring patches.

2.7.1 Lattice Surgery Operations

Lattice surgery [18] implements logical gates between surface code patches by *merging* and *splitting* boundaries. A merge operation effectively measures a joint logical Pauli, e.g. $\bar{Z}_A \bar{Z}_B$, by temporarily joining the Z boundaries of two patches.

All fault-tolerant logical operations can be decomposed into Pauli product measurements (PPMs), enabling a gate-teleportation-based model where magic state injection handles non-Clifford gates.

2.7.2 Litinski’s Game of Surface Codes (GoSC)

Litinski’s *Game of Surface Codes* (GoSC) [19] provides a unified framework for resource-counting lattice surgery computations. The key abstractions are *tiles* (space) and *Tocks* (time), which allow reasoning about fault-tolerant resource costs without descending to physical-qubit-level detail.

Tiles and Patches

The quantum device is modelled as a board partitioned into *tiles*. Each tile corresponds to $\sim 2d^2$ physical qubits (half data, half syndrome) when using a distance- d surface code. A *patch* is a contiguous region of tiles that encodes one logical qubit.

Each patch has two types of boundary:

- **Solid edges** correspond to the logical $\bar{Z}$ operator.
- **Dashed edges** correspond to the logical $\bar{X}$ operator.

The point where a solid and a dashed edge meet is called a *corner* (or X/Z corner). The arrangement of boundaries determines which Pauli operators are accessible for multi-patch measurements. Figure 2.8 shows two common patch orientations.

Tocks

A *Tock* is the unit of time in GoSC, corresponding to d rounds of syndrome extraction (code cycles). 0 Tocks is a special case: it denotes operations with a constant time cost that does not scale with d (but is not necessarily zero physical time). All scheduling costs in this thesis are expressed in Tocks.

Patch Operations

The native operations in the tile-based game, with their time costs, are:

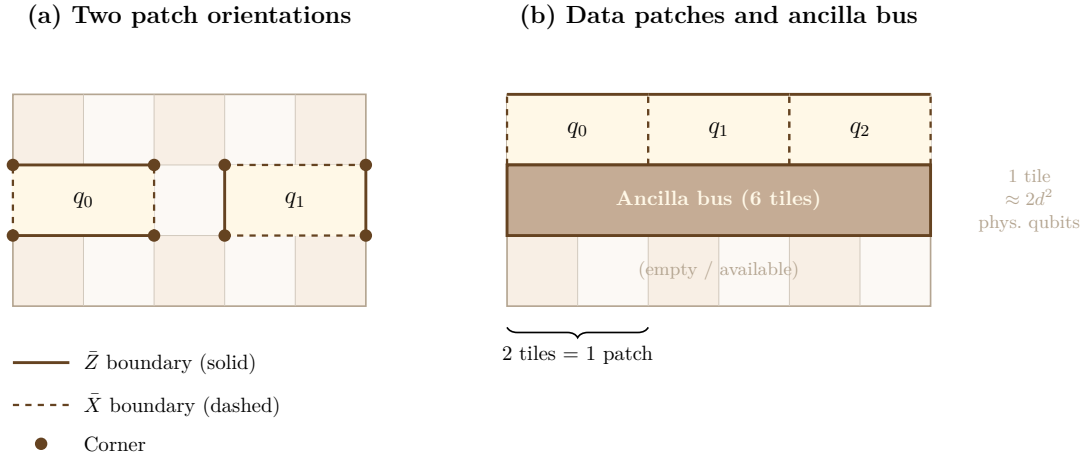


Figure 2.8: Tile-based surface code representation. (a) Two orientations of a one-qubit patch, where solid edges denote $\bar{Z}$ boundaries and dashed edges denote $\bar{X}$ boundaries. (b) Data patches and an ancilla bus on a tile grid. Each tile represents about $2d^2$ physical qubits.

Qubit initialisation (0 Tocks) A one-qubit patch is initialised in a $+1$ eigenstate of $\bar{X}$ or $\bar{Z}$ (i.e. $|+\rangle_L$ or $|0\rangle_L$).

Single-patch measurement (0 Tocks) A patch is measured in the X or Z basis and then removed from the board, freeing its tiles.

Patch deformation / rotation (1 Tock) A patch can expand into adjacent empty tiles (1 Tock) or shrink by releasing tiles (0 Tocks). The X/Z corners can be relocated along the boundary (1 Tock), which is useful for exposing different operators before a multi-patch measurement.

Multi-Pauli Product (MPP) Measurement

A *multi-Pauli product (MPP) measurement* (also called a *parity measurement*) jointly measures an operator $\bigotimes_{i \in S} P_i$ on a subset S of logical qubits, where each $P_i \in \{X, Z, Y\}$.

The key question is: why must this measurement go through an ancilla patch rather than acting directly on the data patches? Measuring each data qubit individually would collapse its state and destroy the encoded logical information. Instead, the goal is to learn only the *joint parity*, whether the qubits collectively are in the $+1$ or -1 eigenspace of $\bigotimes_i P_i$, without revealing any individual qubit's value. The ancilla mediates the measurement. It couples to all target patches through boundary merging, extracts only the parity information, and is then measured out, its tiles returning to the ancilla region for reuse. This leaves the data qubits intact but projected onto the

desired eigenspace. The same principle underlies syndrome extraction in quantum error correction (Section 2.6), where ancilla qubits extract error parities non-destructively.

The procedure is (see Figure 2.9):

- (1) **Initialise** an ancilla patch in $|+\rangle_L$ on empty tiles adjacent to the target patches (0 Tocks).
- (2) **Merge** the ancilla with the relevant boundaries of the target patches and then **split**. The ancilla's $\bar{X}$ or $\bar{Z}$ boundary is merged with each target's corresponding operator boundary. This step projects the target qubits onto the ± 1 eigenspace of $\otimes_i P_i$ and **costs 1 Tock**, regardless of how many qubits participate.
- (3) **Measure and discard** the ancilla patch (0 Tocks). The measurement outcome records the ± 1 eigenvalue.

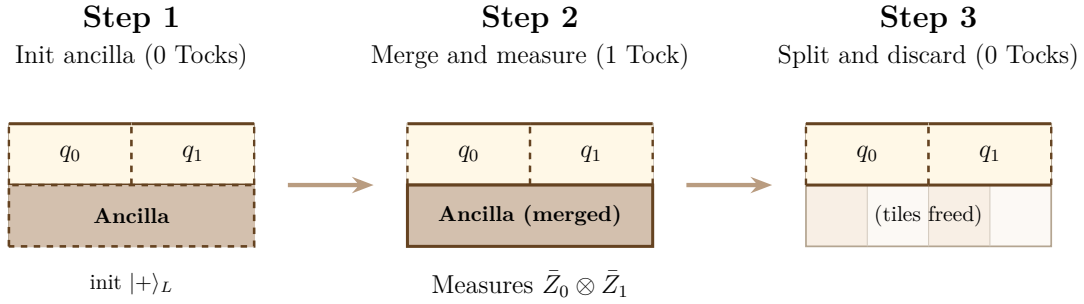


Figure 2.9: An MPP measurement of $\bar{Z}_0 \otimes \bar{Z}_1$. An ancilla patch is initialised, merged with both data patches, and then measured. The entire operation costs 1 Tock regardless of the number of target qubits.

The constant 1-Tock cost of MPP is the key advantage of this operation: a single MPP can simultaneously entangle an arbitrary number of qubits, limited only by the spatial layout (all target patches must be adjacent to the ancilla).

CZ Gate via Lattice Surgery

In GoSC, a logical CZ gate between two patches is implemented through two consecutive merge–measure rounds (see Figure 2.10):

- (1) **Tock 1:** Initialise an ancilla and perform an $\bar{X}\bar{X}$ merge between the two target patches to create a Bell pair (logical entanglement).
- (2) **Tock 2:** Perform a $\bar{Z}\bar{Z}$ measurement between the patches, completing the CZ operation. The ancilla is measured and discarded.

A Pauli byproduct correction (classically tracked in the Pauli frame) may be required based on the measurement outcomes. Each CZ gate therefore costs **2 Tocks** and acts on exactly **two** qubits. It cannot be extended to multi-qubit operations like MPP.

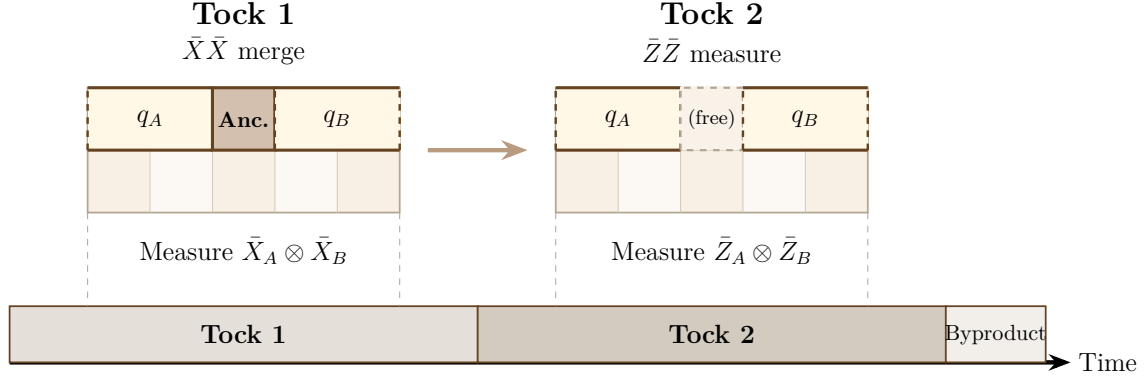


Figure 2.10: CZ gate realised by lattice surgery in two Tocks. In Tock 1 the data patches q_A and q_B merge with an ancilla through an $\bar{X}\bar{X}$ measurement, and in Tock 2 a $\bar{Z}\bar{Z}$ measurement completes the gate and frees the ancilla. Each pairwise CZ gate costs two Tocks.

MPP vs. CZ: Key Trade-off

Table 2.1 summarises the fundamental trade-off between the two operation types. MPP is faster (1 Tock for multi-qubit operations vs. 2 Tocks per pairwise CZ) but requires a shared ancilla bus with spatial adjacency to all participating patches, a more demanding routing requirement. CZ gates are simpler to route (only two patches plus one ancilla) but accumulate more Tocks for graphs with many edges. This trade-off is the central theme of the scheduling analysis in Chapters 4–6.

Property	MPP	CZ
Time cost	1 Tock	2 Tocks
Qubits per op	Arbitrary	Exactly 2
Ancilla routing	Shared bus (complex)	Pairwise (simple)
Parallelism	Via MIS reduction	Via edge colouring

Table 2.1: Comparison of MPP and CZ operations in the GoSC framework.

Space-Time Volume

The total overhead of a GoSC computation is the *space-time volume*:

$$V_{\text{st}} = s \times t \quad (\text{logical}), \quad V_{\text{phys}} \approx s \times t \times 2d^3 \quad (\text{physical qubit-code-cycles}), \quad (2.13)$$

where s is the number of tiles (patches), t is the total Tocks, and each tile-Tock costs about $2d^2$ physical qubits held for d code cycles.

2.8 Distributed Quantum Computing

Classical distributed computing spreads a computation across many networked processors to overcome the memory and speed limits of any single machine. Distributed quantum computing applies the same idea to quantum hardware, but it faces an extra obstacle. Because the no-cloning theorem forbids copying an unknown quantum state, a state cannot simply be sent from one processor to another. Communication between processors must instead rely on shared entanglement.

2.8.1 Architecture

This subsection and the next describe the physical layer of the machine as background. The model of this thesis does not operate at this level of abstraction: it works with logical operations and Tocks, and the assumptions under which the physical layer can be set aside are collected in Section 2.8.3.

In the architecture studied in this thesis, the machine is built from many small quantum processing units, or QPUs. Each QPU holds a single logical qubit, encoded as one surface code patch of the kind described in the previous sections, together with a small number of *communication qubits* whose only job is to generate photonic links to other QPUs.

Entanglement between two QPUs is established by a *memory–interference–memory* (MIM) link, shown in Figure 2.11. A communication qubit in each QPU emits a photon entangled with its own state, the two photons travel to a shared measurement station, and a joint Bell-state measurement there projects the two communication qubits onto a Bell pair. The station performing this measurement is a *Bell-state analyser* (BSA). The BSA also signals whether each attempt succeeded, a property called *heralding*, so a lost photon means trying again rather than a corrupted state. These photonic Bell pairs [20, 21] are the basic resource for communication, taking the place of the wires that carry bits between classical processors.

Figure 2.12 is the same mechanism one level up. The BSAs live inside the optical switch, and the switch routes the photons of any two chosen QPUs onto one of them, so every switched connection is simply an instance of the link in Figure 2.11. Ports count only the external attachment points for QPU channels. The BSAs are internal to the switch fabric and consume no ports, which is why a Bell pair occupies exactly two ports. Because no single chip has to hold every qubit, this design removes the

size limit of a monolithic processor and scales to the large graph states that practical algorithms require.

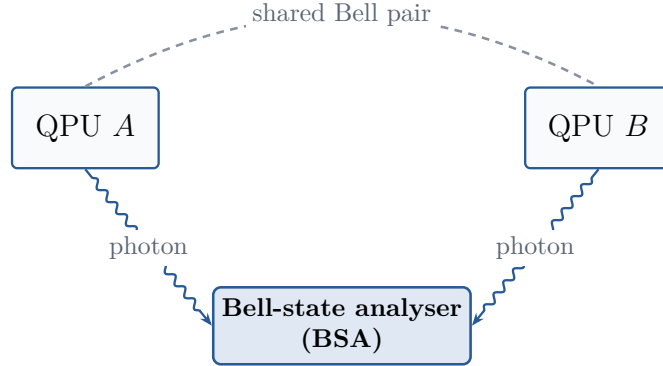


Figure 2.11: An MIM link, the building block of Figure 2.12. Each QPU sends a photon to the Bell-state analyser (BSA), and a successful joint measurement leaves the two QPUs sharing a Bell pair.

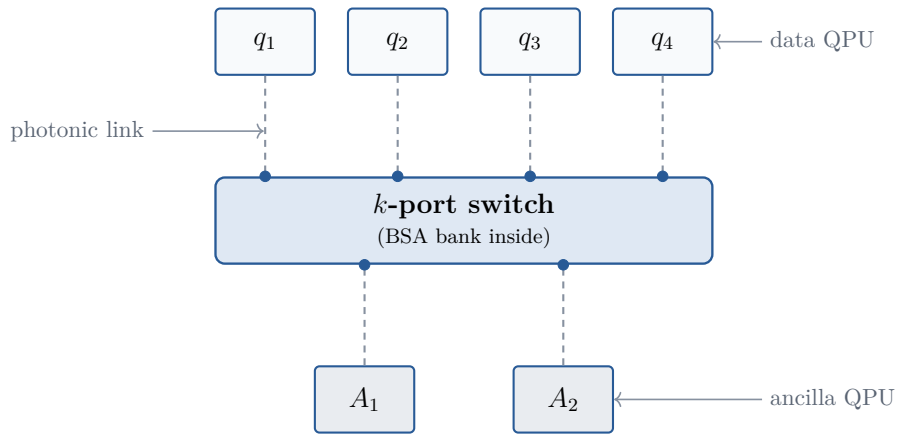


Figure 2.12: The distributed architecture. Each data QPU holds one logical qubit, and a small pool of ancilla QPUs mediates operations. Connecting two switch ports runs the link of Figure 2.11 through the switch, which routes the two photons onto one of its internal BSAs.

2.8.2 Entangling Operations Between QPUs

A shared Bell pair lets two distant QPUs act as if their patches sat next to each other on the same board. A remote lattice surgery merge measures joint stabilizers that span the boundary between the two patches, and on separate devices each of those boundary stabilizer measurements is mediated by shared entanglement. Since the merged boundary is d qubits long and the merge lasts d measurement rounds,

sustaining one remote merge requires a stream of physical Bell pairs, on the order of d pairs per code cycle, delivered by repeating the MIM protocol of Figure 2.11 across the link. By consuming this entanglement, the two QPUs perform the same merge and measure steps used for two adjacent patches [22], so the CZ gates and multi-Pauli-product measurements introduced earlier extend directly to the distributed setting.

2.8.3 Model Assumptions

The model of this thesis operates entirely at the *logical* level: its objects are patches, Tocks, and logical operations, not photons, pulses, or physical error events. The physical layer described above, photon emission, Bell-state analysis, loss, and any purification a platform might require, sits below the model’s abstraction boundary and is presented only as background. The premise of drawing the boundary there is that everything below it affects only constants, such as the duration of a Tock or the entanglement budget a link must supply, whereas the questions this thesis studies, which operations contend for which resources and how deep the resulting schedules are, live entirely above it. The mapping between the two levels rests on four explicit assumptions, stated here once so that every later chapter can rely on them.

1. **Node composition.** A QPU comprises one data patch of $2d^2 - 1$ physical qubits plus a small constant number of communication qubits for link generation. Resource counts in this thesis (N_{phys}) count the patches; the per-QPU communication overhead is a constant factor left outside the counts.
2. **Link budget.** A Tock is long enough for the MIM links engaged in an operation to deliver the Bell pairs that operation consumes. Photon loss is therefore absorbed into the link rate: because generation is heralded, a lost photon costs a repeated attempt within the Tock, never a corrupted state. Loss thus never enters the schedule itself, and this is why the simulator carries no explicit loss model, a choice revisited in Chapter 7.
3. **No purification or distillation.** The link layer is assumed to deliver Bell pairs whose fidelity is already sufficient for lattice surgery at code distance d . The model does not include entanglement purification or distillation; if a given hardware platform required them, they would enlarge the per-Tock Bell-pair budget of assumption 2 but leave the Tock-level schedules unchanged.
4. **Switch independence.** When the machine contains several switches, they operate as mutually unconnected parallel fabrics. Every QPU’s photonic channel can be routed to any switch, but the QPUs engaged in one entangling operation

must all connect through the same switch, because the operation's Bell pairs are formed by BSAs inside a single fabric. Chapter 4 builds the switch-limited scheduling cases on this rule, and Chapter 7 discusses switch-to-switch trunk links, which would relax it, as future work.

2.8.4 Switch Connectivity

The QPUs are not all connected to one another at once. Their photonic channels are routed through optical switches, and a k -port switch can support only k channels at the same time. This places a hard limit on how many QPUs can take part in a joint operation within a single time step. A CZ gate occupies two ports, one for each of the two QPUs involved, while a multi-Pauli-product measurement touching m data qubits occupies $m + 1$ ports, one for every data qubit and one for the shared ancilla. The number of ports k and the number of switches S together cap how much entangling work can proceed in parallel, and this connectivity limit is one of the central factors whose effect on cost this thesis sets out to measure.

2.9 Chapter Summary

This chapter has introduced the foundational concepts of quantum information theory that are required to understand the remainder of the thesis:

- Qubits, superposition, and entanglement.
- Graph states and their stabilizer description.
- The surface code and its resource formula $n_{\text{phys}} = 2d^2 - 1$.
- The GoSC framework, Tocks, and MPP vs. CZ costs.
- Distributed quantum computing with photonic links and k -port switches.

Chapter 3

Problem Statement and Motivation

Graph states are a central primitive in fault-tolerant quantum computing. They appear as resource states for MBQC, as logical Bell pairs for teleportation-based gates, and as the targets of lattice surgery subroutines. Estimating the cost of preparing a graph state on a distributed architecture is therefore a practically important problem that has not been systematically addressed in the literature.

This chapter first reviews the most directly related prior work, then formalises the problem, identifies the three primary cost dimensions, and states the research questions addressed by DQS-Sim.

3.1 Related Work

The problem studied here draws on three strands of prior work, namely distributed quantum architectures, scheduling for fault-tolerant graph state preparation, and automated resource estimation. Distributed quantum computing was proposed early on by Cirac, Ekert, Huelga, and Macchiavello [23], and developed into a quantum-multicomputer architecture by Van Meter [24]. Wehner, Elkouss, and Hanson [25] later set out a hierarchy of quantum network capabilities whose most capable level, full quantum computing networks with local memories and gates, is the setting of this thesis. The architecture itself is described in Section 2.8. The preparation of graph states has itself been studied as a circuit-level optimisation problem, for instance minimising the number of entangling gates needed [26], and as a compilation target that turns a high-level algorithm into the graph state it consumes [27]. Closest on the distributed side, the Quantum Hamlets framework [28] partitions large algorithmic graph states across multiple QPUs so as to minimise the Bell pairs a static partition consumes, and identifies dynamic distributed compilation as an open problem. The two strands most directly related to the present work, the monolithic scheduler this thesis extends and the resource-estimation tools it complements, are reviewed below.

3.1.1 The Substrate Scheduler

The immediate predecessor of this work is the Substrate Scheduler developed by Liu and colleagues at Keio University SFC [1]. That work scheduled graph state preparation on a monolithic single-processor surface code device using the GoSC framework. It implements the MPP-based approach, which appears as Case 1 in this thesis, through three phases. A maximum independent set of the graph is first computed, so that its vertices can be initialised in $|+\rangle$ at no measurement cost. The remaining stabilizer generators are then laid out along the substrate so that vertices connected in the graph sit near one another, and finally assigned to time slots by a greedy interval schedule in which no two simultaneously scheduled generators share support.

Liu’s model assumes a single surface code substrate with no inter-processor communication overhead, an unlimited supply of ancilla patches, and no constraint on how many operations may run at once. These assumptions suit a single large quantum computer but do not capture a distributed machine, where each QPU holds one logical qubit, communication is mediated by photonic links, and switches impose a port-count limit. This thesis extends the Substrate Scheduler’s approach to that distributed setting. DQS-Sim is an independent implementation written from scratch. Case 1 re-implements the three-phase MPP pipeline of the original and reproduces its behaviour, and the CZ-based, ancilla-limited, and switch-limited cases then add what the monolithic model does not address.

3.1.2 Resource Estimation Frameworks

Several automated tools estimate the resource cost of fault-tolerant quantum algorithms. Microsoft’s resource-estimation framework [29] compiles a high-level quantum algorithm down to physical qubit counts and runtime estimates using the surface code, and the open-source BenchQ pipeline [30] offers comparable functionality. Both target single-processor algorithms and do not model the inter-processor communication that a distributed machine introduces. DQS-Sim occupies a complementary niche: it estimates the Tock-level scheduling cost of the graph state preparation sub-problem under explicit distributed constraints, modelling the choice between MPP and CZ operations, the size of the ancilla pool, and the port limit of the optical switches. Table 3.1 positions DQS-Sim relative to these frameworks.

Tool	Distributed	Switch model	MPP & CZ	Graph state focus
Microsoft estimator [29]	✗	✗	✗	✗
BenchQ [30]	✗	✗	✗	✗
Substrate Scheduler [1]	✗	✗	MPP only	✓
Quantum Hamlets [28]	✓	✗	✗	✓
DQS-Sim (this work)	✓	✓	✓	✓

Table 3.1: Comparison of resource estimation frameworks for fault-tolerant quantum computing.

3.2 Formal Problem Statement

Problem 3.1 (Distributed Graph State Cost Estimation). Given a target graph $G = (V, E)$ with $n = |V|$ vertices and $m = |E|$ edges, a surface code distance d , and hardware parameters (k, S) (ports per switch, number of switches), compute:

- The schedule depth T (in Tocks) achieved for preparing $|G\rangle$ under each of the five scheduling cases defined below, together with its theoretical bounds.
- The peak number of ancilla QPUs required simultaneously.
- The total number of physical qubits $N_{\text{phys}} = (n_{\text{data}} + n_{\text{ancilla}}^{\text{peak}}) \times (2d^2 - 1)$.

The five cases combine the choice of entangling operation with the hardware constraints, listed here from the most idealised to the most realistic.

- **Case 1 (MPP, unlimited ancillas).** Stabilizer measurements with no limit on ancillas or connectivity.
- **Case 2 (CZ, unlimited ancillas).** CZ gates with no limit on ancillas or connectivity.
- **Case 3 (CZ, k_a ancilla QPUs).** CZ gates restricted to a fixed pool of k_a ancilla QPUs.
- **Case 4 (MPP, switch-limited).** Stabilizer measurements routed through S switches of k ports each.
- **Case 5 (CZ, switch-limited).** CZ gates routed through S switches of k ports each.

Cases 1 and 2 are the idealised baselines, Case 3 adds the ancilla-pool limit, and Cases 4 and 5 add the switch connectivity constraint.

The switch parameters (k, S) obey the connectivity rule fixed by the model assumptions of Section 2.8.3: the S switches are mutually unconnected parallel fabrics, every QPU's photonic channel can be routed to any of them, and all the QPUs engaged in one entangling operation must connect through the same switch, since the operation's Bell pairs are produced by Bell-state analysers inside a single fabric. Consequently k limits the size and throughput of individual operations, while S limits only how many operations can run side by side; the two are independent hardware inputs, and Chapter 6 measures their effects separately.

3.2.1 Notation

Table 3.2 collects the symbols used throughout the remaining chapters, in one place, in the order they typically appear.

Symbol	Meaning
$G = (V, E), n, m$	target graph, its vertex count, its edge count
$\Delta(G)$	maximum degree of G
$\alpha(G)$	size of a maximum independent set of G
$g_v, \text{supp}(g_v)$	stabilizer generator of vertex v , and the qubits it touches
$\mathcal{C}$	conflict graph of the measured generators
$\omega(\mathcal{C}), \chi(\mathcal{C})$	clique number and chromatic number of $\mathcal{C}$
$\chi'(G), C_i$	chromatic index of G ; its i -th colour class
d	surface code distance
Tock	d error-correction rounds, the unit of schedule time
$T_1, \dots, T_5$	schedule depth in Tocks under Cases 1 to 5
k_a	ancilla-pool size (Case 3)
k, S	ports per switch; number of switches (Cases 4 and 5)
$\ell = S\lfloor k/2 \rfloor$	machine-wide CZ throughput, gates per round
N_{phys}	total physical qubits, $(n_{\text{data}} + n_{\text{ancilla}}^{\text{peak}})(2d^2 - 1)$

Table 3.2: Notation used throughout the thesis. Graph-theoretic terms are defined in Section 2.4.1, and the hardware model behind d , Tocks, k , and S in Chapter 2.

3.3 The Three Cost Dimensions

Three orthogonal dimensions determine the total resource cost of distributed graph state preparation:

3.3.1 Dimension 1: Operation Type, MPP versus CZ

The GoSC framework offers two mechanisms for implementing graph state edges:

MPP-based (Cases 1, 4): Measure the multi-qubit stabilizer generator $g_i = X_i \otimes_{j \in N(i)} Z_j$ for each vertex i . The set of qubits a generator acts on, vertex i together with its neighbours, is called its *support* and written $\text{supp}(g_i)$. An MPP measurement costs 1 Tock regardless of the support size. Non-conflicting generators can be measured simultaneously.

CZ-based (Cases 2, 3, 5): Apply a physical CZ gate for every edge $(i, j) \in E$. Each CZ takes 2 Tocks and is confined to two adjacent QPUs. CZ gates on vertex-disjoint edges can run in parallel.

The MPP approach is generally faster (fewer Tocks per unit of structure prepared) but requires routing multi-qubit Pauli operations through the photonic network, which may introduce overhead not captured in the basic Tock count. CZ-based preparation is simpler to implement but scales with the number of edges.

For the complete graph K_5 (5 vertices, 10 edges), our simulator measures 4 Tocks for MPP against 14 Tocks for CZ, making MPP $3.5\times$ faster on this graph.

3.3.2 Dimension 2: Ancilla QPU Pool Size

In CZ-based preparation, an ancilla QPU mediates the lattice surgery merges that implement each CZ gate. The number of CZ gates that can run in parallel is therefore bounded by the available ancilla QPUs:

- **Unlimited ancillas (Case 2):** All edges in a colour class execute in one round of 2 Tocks. Peak ancilla demand = maximum colour-class size.
- k_a **ancillas (Case 3):** Each colour class C_i is split into sub-batches of $\leq k_a$ edges, each sub-batch taking 2 Tocks. Schedule depth = $2 \sum_i \lceil |C_i|/k_a \rceil$.

As k_a shrinks from the size of the largest colour class, where the limit stops binding, down to 1, where every gate runs alone, the Tock count rises from $2\chi'(G)$ to $2|E|$, a slowdown of up to $|E|/\chi'(G)$.

3.3.3 Dimension 3: Switch Connectivity

A k -port switch constrains how many QPUs can simultaneously interact. Specifically:

- A CZ gate uses 2 photonic ports on the switch (one per QPU). Thus at most $S \lfloor k/2 \rfloor$ simultaneous CZ gates are possible across S switches.

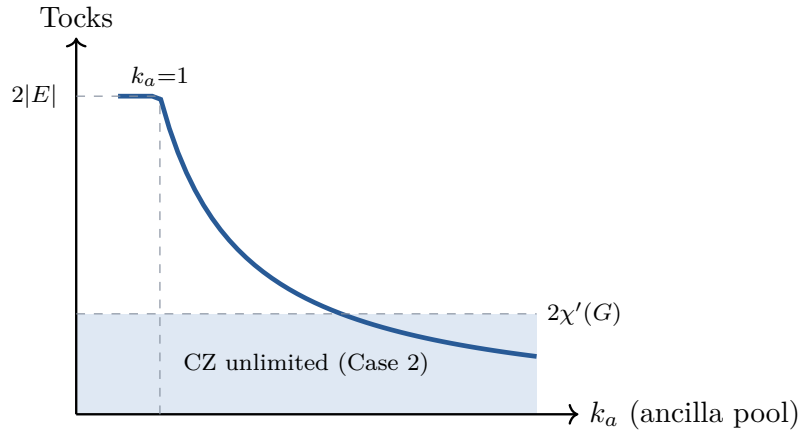


Figure 3.1: Schedule depth against ancilla pool size k_a for Case 3. At $k_a = 1$ every gate runs alone and the depth is $2|E|$. Larger pools lower the depth until it settles at $2\chi'(G)$, the depth of the unlimited-ancilla Case 2.

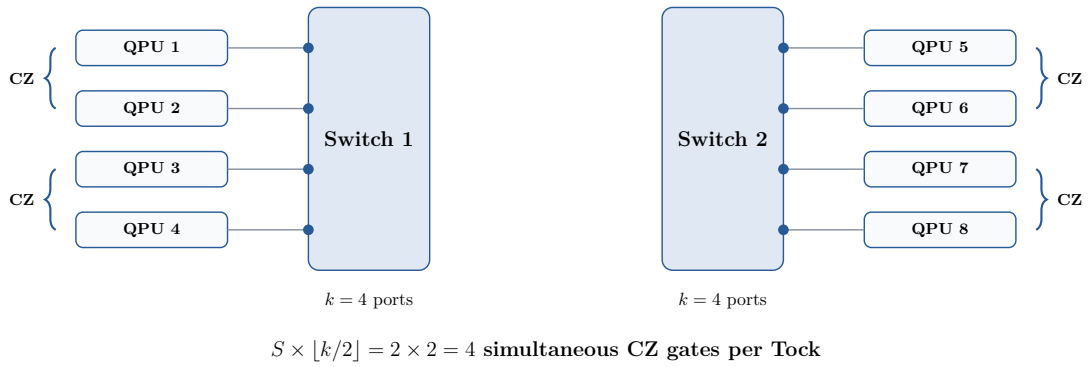


Figure 3.2: Two $k = 4$ port switches, each allowing $\lfloor 4/2 \rfloor = 2$ simultaneous CZ pairs. In total, $S \lfloor k/2 \rfloor = 4$ CZ gates can execute per Tock.

- An MPP measurement touching m data qubits needs every one of them connected to one shared ancilla, all through the same switch. It therefore occupies $m + 1$ ports, one per data qubit and one for the ancilla, and is only possible at all when $m + 1 \leq k$.

When the graph's maximum degree is large relative to k , this constraint starts to matter, and it affects the two operations differently. For CZ the effect is throughput. Each round of gates is split into smaller waves that fit through the switch, and the extra waves stretch the schedule. For MPP the effect is harsher. A single measurement can never be split, so fewer measurements share each Tock, and a measurement needing more than k ports cannot run at all. Chapter 6 measures both effects. Figure 3.2 illustrates the CZ throughput limit, with two four-port switches carrying at most four CZ gates in one Tock.

3.4 Research Questions

This thesis addresses the following research questions:

- RQ1. **Comparative cost:** How do MPP-based and CZ-based graph state preparation compare in terms of Tocks, peak ancilla QPUs, and total physical qubits for representative graph families (cycles, complete graphs, stars, grids)?
- RQ2. **Ancilla sensitivity:** How does the schedule depth of CZ-based preparation degrade as the ancilla pool is restricted from unlimited to a single QPU? Is there a practical crossover point?
- RQ3. **Switch-constraint impact:** By what factor does a k -port switch constraint increase the schedule depth for MPP and CZ methods, as a function of graph degree $\Delta(G)$?
- RQ4. **Physical resource totals:** Accounting for surface code overhead ($2d^2 - 1$ physical qubits per QPU), what are the total physical qubit requirements for each case?

3.5 Scope and Assumptions

The following assumptions define the scope of this work:

1. **One logical qubit per QPU.** Each QPU holds exactly one logical qubit encoded in a distance- d surface code patch.
2. **Tock-level abstraction.** All costs are expressed in Tocks (one Tock = d rounds of syndrome extraction). Sub-Tock photon-loss errors and Bell-pair generation retries are not modelled.
3. **GoSC operation costs.** CZ = 2 Tocks; MPP = 1 Tock, regardless of support size (subject to switch constraints).
4. **Graph state target.** We focus on the graph state preparation sub-problem. Downstream MBQC computation costs are out of scope.
5. **Identical, independent switches.** The machine may contain any number S of switches, and Cases 4 and 5 study exactly that setting, but all switches share the same port count k and are not connected to one another. Switches with differing port counts, and switch-to-switch trunk links, are left for future work (Chapter 7).

3.6 Chapter Summary

This chapter has:

- Stated the distributed graph state cost estimation problem formally.
- Identified the three cost dimensions: operation type, ancilla pool size, and switch connectivity.
- Reviewed prior work, including the Substrate Scheduler and existing resource estimation frameworks.
- Formulated four concrete research questions.
- Defined the scope and key assumptions.

Chapter 4 describes the DQS-Sim simulator that addresses these questions.

Chapter 4

Method: DQS-Sim

DQS-Sim (*Distributed Quantum Scheduling Simulator*) is a Python package for estimating the fault-tolerant cost of graph state preparation in a distributed quantum computing architecture. It represents target graphs and computes the graph properties used by the schedulers with the NetworkX library [31]. This chapter describes the system architecture, the five scheduling algorithms (Cases 1–5), and the physical resource calculation.

4.1 System Architecture

4.1.1 Module Overview

DQS-Sim is organised into four modules:

schedulers.py Implements the five scheduling algorithms as `SubstrateScheduler` (Case 1), `CZScheduler` (Case 2), `CZSchedulerFixedAncilla` (Case 3), `MPPSchedulerLimited` (Case 4), and `CZSchedulerLimited` (Case 5).

cost_estimator.py Aggregates scheduler outputs into a `ResourceReport` dataclass containing all Tock counts, peak ancilla QPUs, and physical qubit totals.

topology.py Defines the `KPortSwitch` dataclass encoding switch hardware parameters (k , S , n_{QPU}).

visualizer.py Generates Gantt charts, heatmaps, occupancy strips, 2-D layout diagrams, and comparison dashboards.

4.1.2 Data Flow

Figure 4.1 traces a run through the four modules, from the target graph and hardware parameters to the rendered outputs.

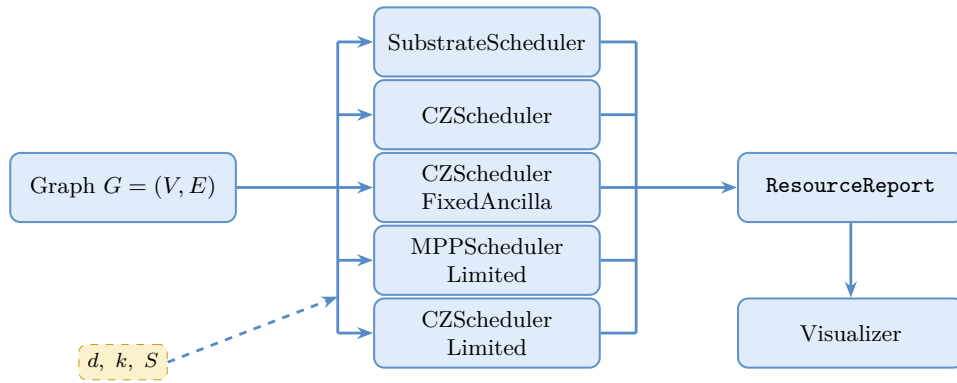


Figure 4.1: DQS-Sim data flow. A target graph and hardware parameters are fed to all five schedulers; outputs are aggregated into a **ResourceReport** and rendered by the Visualizer.

4.1.3 Command-Line Interface

DQS-Sim exposes a CLI via `main.py`:

```

1 python main.py --graph cycle --n 8 --case all \
2             --ancillas 2 --code-distance 7 \
3             --switch-k 4 --switches 2 --save-all Results/

```

DQS-Sim schedules *any* graph, including algorithm-derived graph states. Built-in generators cover the families used in this thesis: `path`, `cycle`, `star`, `complete`, `grid`, `random`, `petersen`, `hypercube`.

4.2 Case 1: MPP Scheduling, Unlimited Ancillas

Case 1 re-implements the monolithic Substrate Scheduler’s algorithm [1] from scratch and extends it to serve as the MPP baseline in the distributed model.

4.2.1 Algorithm

The algorithm has three phases:

Phase 1 — MIS Reduction. Compute a Maximum Independent Set $I \subseteq V$. Vertices in I are initialised in $|+\rangle$ at zero Tock cost. Only the vertex cover $V \setminus I$ requires generator measurements. For $n \leq 20$ an exact MIS is computed via exhaustive search; for $n > 20$ a greedy degree-ordering heuristic is used.

Phase 2 — Vertex-to-Qubit Mapping. The scheduler next fixes where each logical qubit physically sits. The QPUs are arranged as a linear substrate with one data position and one ancilla position per qubit, following the substrate model of the monolithic scheduler [1], and a mapping $\pi : V \rightarrow \{0, \dots, n-1\}$ assigns each graph vertex to a substrate position. A good mapping seats vertices that are connected in the graph near one another on the substrate, so that the supports of most generators stay compact. Finding a truly optimal seating is NP-hard, so DQS-Sim uses a standard shortcut. It computes a layout score for every vertex, chosen so that connected vertices receive similar scores, and then simply sorts the vertices by score. The scores are chosen by a standard eigenvector computation whose full details live in the DQS-Sim source; what matters for the schedule is only the resulting order. On a path graph, for example, the scores rise steadily from one end to the other, so sorting recovers the path’s own order, the ideal layout.

Phase 3 — Greedy Interval Scheduling. The mapping matters because of how a measurement occupies the ancilla row. Measuring g_i requires an ancilla region that connects every qubit in its support $\text{supp}(g_i)$, the qubits the generator acts on, and on a linear substrate that region is the contiguous interval

$$[L_i, R_i], \quad L_i = \min_{v \in \text{supp}(g_i)} \pi(v), \quad R_i = \max_{v \in \text{supp}(g_i)} \pi(v), \quad (4.1)$$

spanning *all* ancilla positions between the leftmost and rightmost support qubit. Positions inside the interval are blocked even when their data qubit is not involved: if a generator touches the qubits at positions 1, 2, and 4, its ancilla region occupies positions 1 through 4, and the ancilla at position 3 is unavailable to any other measurement in that Tock. Two measurements can therefore share a Tock only if their intervals are disjoint, a strictly stronger requirement than having disjoint supports. The scheduler packs the intervals greedily in earliest-deadline order. Algorithm 1 gives the full procedure.

Algorithm 1 MPP Greedy Interval Schedule (Case 1)**Input:** Graph $G = (V, E)$, mapping π **Output:** Schedule $\mathcal{T} = [\tau_1, \tau_2, \dots]$, each slot τ_t a set of generators

```

1:  $I \leftarrow \text{MIS}(G)$ 
2: for all  $i \in V \setminus I$  do
3:    $[L_i, R_i] \leftarrow$  ancilla interval of  $g_i$  under  $\pi$  ▷ Eq. (4.1)
4: end for
5: Sort blocks by  $(R_i, L_i)$  ▷ earliest deadline first
6: while unscheduled blocks remain do
7:   Open a new slot  $\tau$ ;  $r \leftarrow -1$ 
8:   for all blocks  $g_i$  in sorted order do
9:     if  $L_i > r$  then ▷ interval clears the occupied prefix
10:      add  $g_i$  to  $\tau$ ;  $r \leftarrow R_i$ 
11:    end if
12:  end for
13: end while
14: return  $\mathcal{T}$ 

```

4.2.2 Resource Metrics

Throughout this chapter, T_i denotes one quantity only, the schedule depth in Tocks achieved by Case i . For Case 1, every slot of the schedule lasts exactly one Tock, because each MPP costs one Tock, so the depth is simply the number of slots. The substrate provides a single shared ancilla row, counted as one ancilla QPU, and the physical total counts the n data patches plus that one ancilla patch:

$$T_1 = |\mathcal{T}| \quad (\text{number of schedule slots}), \quad (4.2)$$

$$\text{peak ancilla QPUs}_1 = 1 \quad (\text{shared ancilla bus}), \quad (4.3)$$

$$N_{\text{phys},1} = (n + 1) \times (2d^2 - 1). \quad (4.4)$$

The lower bound is $T_1 \geq \omega(\mathcal{C})$, the clique number of the support-conflict graph, since overlapping supports imply overlapping intervals. Interval blocking can push the schedule above this bound, because two generators with disjoint supports may still have overlapping ancilla intervals; the verification layer checks the measured depth against the greedy colouring bound $\Delta(\mathcal{C}) + 1$ as a sanity bound (Chapter 5).

4.3 Case 2: CZ Scheduling, Unlimited Ancillas

Case 2 prepares $|G\rangle$ by applying a CZ gate for each edge $(i, j) \in E$, parallelising via edge colouring.

4.3.1 Algorithm

Algorithm 2 CZ Edge-Colouring Schedule (Case 2)

Input: Graph $G = (V, E)$

Output: Colour classes $C_1, \dots, C_{\chi'}$ (each a matching)

- 1: Initialise $colour[e] \leftarrow \emptyset$ for all $e \in E$
 - 2: **for all** $e = (u, v) \in E$ in degree-decreasing order **do**
 - 3: $F_u \leftarrow$ set of colours used by edges incident to u
 - 4: $F_v \leftarrow$ set of colours used by edges incident to v
 - 5: $c \leftarrow \min(\mathbb{N} \setminus (F_u \cup F_v))$ $\triangleright$ smallest available colour
 - 6: $colour[e] \leftarrow c$
 - 7: **end for**
 - 8: **return** colour classes $C_c = \{e : colour[e] = c\}$
-

Algorithm 2 takes the bare target graph and returns its edges grouped into colour classes $C_1, \dots, C_{\chi'}$; each class is a matching, so each class is one two-Tock round of simultaneous CZ gates, and the number of classes is the greedy estimate of $\chi'(G)$. It runs in $O(m \log \Delta)$ time. It is not optimal. In the worst case it uses up to $2\Delta - 1$ colours where Vizing's theorem guarantees $\Delta + 1$ always suffice, so the greedy round count upper-bounds the true $\chi'(G)$. We accept this trade deliberately. Greedy colouring is simple and fast on arbitrary graphs, and the verification layer computes the Vizing window on every run and flags any colouring that exceeds $\Delta + 1$, so the gap is always measured rather than hidden. On the structured families of Chapter 6 the greedy result often meets the optimum anyway, as on C_6 where it finds the perfect two colours, and where it does not, as on K_5 with seven colours against an optimum of five, the overhead is reported explicitly. The colouring is also a plug-in subroutine, and Chapter 7 notes that substituting an optimal colouring would only tighten the CZ depths.

4.3.2 Resource Metrics

Each colour class is a matching (vertex-disjoint edges) and executes in one round of 2 Tocks, so the depth T_2 is two Tocks per class. Every gate running in a round needs its own ancilla QPU, so the peak ancilla demand is the size of the largest class:

$$\chi'(G) = \text{number of colour classes}, \quad (4.5)$$

$$T_2 = 2\chi'(G), \quad (4.6)$$

$$\text{peak ancilla QPUs}_2 = \max_c |C_c|, \quad (4.7)$$

$$N_{\text{phys},2} = (n + \max_c |C_c|) \times (2d^2 - 1). \quad (4.8)$$

4.4 Case 3: CZ Scheduling, Fixed Ancilla Pool

Case 3 extends Case 2 by restricting the ancilla pool to k_a QPUs, where k_a is a user-specified hardware parameter.

4.4.1 Algorithm

Algorithm 3 splits each colour class C_i from Case 2 into sub-batches of at most k_a edges. Figure 4.2 illustrates how the splits stretch the schedule.

Algorithm 3 CZ Fixed-Ancilla Schedule (Case 3)

Input: Colour classes $C_1, \dots, C_{\chi'}$ (produced by Algorithm 2); ancilla pool size k_a

Output: Sub-batch sequence $B_1, B_2, \dots$

- 1: $\mathcal{B} \leftarrow []$
 - 2: **for all** colour class C_i **do**
 - 3: Partition C_i into consecutive sub-batches of size $\leq k_a$: $C_i = B_{i,1} \cup \dots \cup B_{i,s_i}$,
 $|B_{i,j}| \leq k_a, s_i = \lceil |C_i|/k_a \rceil$
 - 4: Append $B_{i,1}, \dots, B_{i,s_i}$ to $\mathcal{B}$
 - 5: **end for**
 - 6: **return** $\mathcal{B} = [B_1, B_2, \dots]$, the sub-batches in execution order
-

Algorithm 3 consumes the colour classes produced by Algorithm 2 and returns the ordered list of sub-batches $\mathcal{B}$; each sub-batch contains at most k_a gates, runs in one two-Tock round with one ancilla QPU per gate, and the batches execute strictly in sequence, so the schedule depth is twice their count.

4.4.2 Resource Metrics

$$n_{\text{batches}} = \sum_{i=1}^{x'} \left\lceil \frac{|C_i|}{k_a} \right\rceil, \quad (4.9)$$

$$T_3 = 2 \cdot n_{\text{batches}}, \quad (4.10)$$

$$\text{peak ancilla QPUs}_3 = k_a, \quad (4.11)$$

$$N_{\text{phys},3} = (n + k_a) \times (2d^2 - 1). \quad (4.12)$$

Special cases.

- $k_a \geq \max_i |C_i|$: Case 3 reduces to Case 2 ($T_3 = T_2$).
- $k_a = 1$: fully sequential; $T_3 = 2|E|$ and peak ancilla QPUs = 1.

Crossover with Case 1. For sparse graphs, Case 1 (MPP) can be cheaper than Case 3 (CZ, restricted) even for modest k_a , and Chapter 6 measures where that boundary falls.

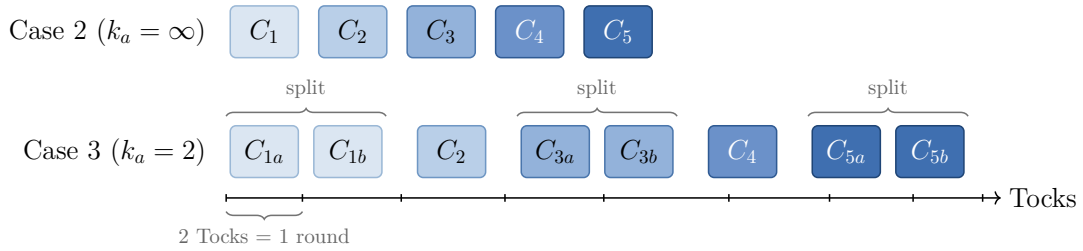


Figure 4.2: Ancilla splitting in Case 3. With unlimited ancillas (Case 2, top row) each colour class runs as a single batch. With a pool of only $k_a = 2$ (bottom row), any class holding more than k_a edges is split into smaller batches. Every batch takes 2 Tocks, so each split makes the schedule longer.

4.5 Case 4: MPP Scheduling, Limited Switch Connectivity

Case 4 extends the MPP approach (Case 1) to the distributed setting with S switches, each having k ports. Following the model assumptions of Section 2.8.3, the switches are identical and independent. No switch connects to another, and any QPU can attach to any free port of any switch. A switch can host several measurements in the same Tock, each bringing its own ancilla, as long as their combined port demand stays within its k ports.

4.5.1 Switch Feasibility Constraint

An MPP measurement of generator g_i with $|\text{supp}(g_i)| = s_i$ requires:

- s_i ports for the data QPUs,
- 1 port for the ancilla QPU (shared bus).

Therefore the constraint per switch is $s_i + 1 \leq k$, i.e. the generator's support must fit within $k - 1$ data ports.

4.5.2 Algorithm

Algorithm 4 MPP Switch-Limited Schedule (Case 4)

Input: Generators $\{g_i\}$; S switches of k ports each

Output: Batches $B_1, B_2, \dots$, each assigned to switches

- 1: Build conflict graph $\mathcal{C}$: edge (i, j) if $\text{supp}(g_i) \cap \text{supp}(g_j) \neq \emptyset$
 - 2: Compute greedy colouring of $\mathcal{C}$: colour classes $\mathcal{C}_1, \dots, \mathcal{C}_\ell$
 - 3: **for all** colour class $\mathcal{C}_r$ (non-conflicting generators) **do**
 - 4: Use first-fit bin-packing to assign generators in $\mathcal{C}_r$ to switches:
 - 5: Constraint per switch: $\sum_j (s_j + 1) \leq k$ over its assigned generators g_j
 - 6: **if** a generator does not fit in any switch **then**
 - 7: Open a new time sub-slot (carry forward to next batch)
 - 8: **end if**
 - 9: **end for**
 - 10: **return** $\mathcal{B} = [B_1, B_2, \dots]$, the batches with their switch assignments
-

Algorithm 4 takes the cover generators and the switch parameters and returns a sequence of batches, each batch being a set of mutually non-conflicting generators together with an assignment of each generator to one switch; every batch executes in one Tock. A switch may host several measurements at once. Each occupies s_j data ports plus one ancilla port of its own, so a six-port switch can carry one degree-4 measurement or two degree-1 measurements in the same Tock. The first-fit bin-packing is applied per colour class to minimise the number of sub-slots (and hence Tocks) needed when several generators must share the S switches. If any single generator has $|\text{supp}(g_i)| + 1 > k$, no assignment exists and the instance is reported infeasible, the feasibility rule of Section 4.5.

4.6 Case 5: CZ Scheduling, Limited Switch Connectivity

Case 5 extends CZ scheduling (Case 2) to the switch-limited setting.

4.6.1 Switch Throughput

Each switch with k ports supports $\lfloor k/2 \rfloor$ simultaneous CZ gates (2 ports per CZ). With S switches in parallel, the maximum CZ gates per Tock is:

$$\text{CZ per Tock} = S \left\lfloor \frac{k}{2} \right\rfloor. \quad (4.13)$$

The switch topology is the same as in Case 4. The S switches are identical, independent, and not connected to one another, and any QPU can attach to any free port of any switch (Section 2.8.3). This means no path selection or routing problem arises. A CZ gate occupies two ports of one switch, the Bell pair is produced by a BSA inside that switch, and no operation ever spans more than one switch. The scheduler therefore only counts capacity. It fills the $S \lfloor k/2 \rfloor$ gate slots of each Tock in edge-colouring order, and which physical switch serves which gate is irrelevant to the cost.

4.6.2 Algorithm

Algorithm 5 applies the same splitting as Case 3, with the machine-wide throughput ℓ taking the place of the pool size k_a .

Algorithm 5 CZ Switch-Limited Schedule (Case 5)

Input: Colour classes $C_1, \dots, C_{\chi'}$ (produced by Algorithm 2); S switches of k ports each

Output: Sub-batch sequence $B_1, B_2, \dots$

- 1: $\ell \leftarrow S \lfloor k/2 \rfloor$ $\triangleright$ CZ gates that fit per round
 - 2: $\mathcal{B} \leftarrow []$
 - 3: **for all** colour class C_i **do**
 - 4: Partition C_i into $\lceil |C_i|/\ell \rceil$ sub-batches of size $\leq \ell$ and append them to $\mathcal{B}$
 - 5: **end for**
 - 6: **return** $\mathcal{B} = [B_1, B_2, \dots]$, the sub-batches in execution order
-

4.6.3 Resource Metrics

The switch throughput ℓ takes over the role the pool size k_a played in Case 3, so the depth is again two Tocks per wave and the helpers in use never exceed ℓ :

$$\ell = S \left\lfloor \frac{k}{2} \right\rfloor, \quad (4.14)$$

$$T_5 = 2 \sum_{i=1}^{x'} \left\lceil \frac{|C_i|}{\ell} \right\rceil, \quad (4.15)$$

$$\text{peak ancilla QPUs}_5 = \ell. \quad (4.16)$$

Relationship to Case 3. Case 5 is formally identical to Case 3 with $k_a \leftarrow \ell = S\lfloor k/2 \rfloor$. The switch constraint plays the same role as the ancilla pool constraint, but is derived from hardware parameters rather than being user-specified.

4.7 Physical Resource Calculation

4.7.1 Physical Qubits per QPU

Each logical QPU is encoded in a distance- d surface code patch with

$$n_{\text{phys}}(d) = 2d^2 - 1 \quad (4.17)$$

physical qubits. Worked examples in this thesis use $d = 7$, giving $n_{\text{phys}} = 97$. The value is DQS-Sim’s default code distance and is illustrative rather than tuned to a particular error budget. Since the distance only sets the per-QPU constant, every count in this chapter converts to any other distance by re-evaluating $2d^2 - 1$, and the CLI accepts any distance through the `--code-distance` flag.

4.7.2 Total Physical Qubit Count

The total physical qubit requirement for each case is:

$$N_{\text{phys}}^{(c)} = (n_{\text{data}} + n_{\text{ancilla}}^{\text{peak},(c)}) \times n_{\text{phys}}(d), \quad (4.18)$$

where $n_{\text{data}} = |V|$ and $n_{\text{ancilla}}^{\text{peak},(c)}$ is the peak ancilla QPU count for case c .

Table 4.1 summarises the formula for each case.

4.7.3 Space-Time Cost

The GoSC space-time product (in logical qubit-Tocks) is:

$$\text{ST} = (n + n_{\text{ancilla}}^{\text{peak}}) \times T. \quad (4.19)$$

Case	Tocks	Peak ancilla QPUs	N_{phys}
1	T_1 (measured)	1	$(n + 1)(2d^2 - 1)$
2	$2\chi'(G)$	$\max_i C_i $	$(n + \max_i C_i)(2d^2 - 1)$
3	$2\sum_i \lceil C_i /k_a \rceil$	k_a	$(n + k_a)(2d^2 - 1)$
4	T_4 (measured)	g_{max}	$(n + g_{\text{max}})(2d^2 - 1)$
5	$2\sum_i \lceil C_i /\ell \rceil$	ℓ	$(n + \ell)(2d^2 - 1)$

Table 4.1: Resource formulae for the five DQS-Sim cases. Here $\ell = S \lfloor k/2 \rfloor$ is the CZ throughput of the switches, and g_{max} is the largest number of measurements scheduled in a single Tock in Case 4. Each concurrent measurement uses one ancilla QPU, and measurements are packed across the S switches with each switch hosting measurements whose combined port demand, data qubits plus one ancilla port per measurement, fits within its k ports.

Converting to physical qubit-code-cycles (accounting for d syndrome rounds per Tock):

$$\text{ST}_{\text{phys}} = N_{\text{phys}} \times T \times d. \quad (4.20)$$

This is the most comprehensive single-number summary of the computation cost.

4.8 Implementation Details

4.8.1 Graph Construction

DQS-Sim uses NetworkX for graph construction, and its schedulers accept any NetworkX graph, algorithm-derived graph states included. Built-in generators cover the common families for convenience, and the experiments of Chapter 6 draw their instances from these generators plus the incidence graphs constructed for the MPP-versus-CZ study.

4.8.2 MIS Solver

For $n \leq 20$, the MIS is computed exactly using a branch-and-bound search. For $n > 20$, a greedy algorithm is applied: repeatedly select the minimum-degree vertex (break ties by index), add it to the MIS, and remove its neighbours.

4.8.3 Edge Colouring

The greedy edge colouring processes edges in decreasing degree order. For each edge (u, v) , it assigns the smallest colour not used by any edge incident to u or v . The

algorithm is $O(m \log \Delta)$ and uses at most $2\Delta - 1$ colours, which upper-bounds the optimal $\chi' \leq \Delta + 1$.

4.8.4 Verification

After each scheduling run, DQS-Sim verifies:

- Case 1: $\omega(\mathcal{C}) \leq T_1 \leq \Delta(\mathcal{C}) + 1$.
- Case 2: $\Delta(G) \leq \chi' \leq 2\Delta(G) - 1$ (greedy bound; the result is flagged as suboptimal when it exceeds the Vizing optimum $\Delta + 1$).
- Case 3: $T_3 \geq T_2$ (more or equal Tocks under ancilla restriction).

These checks can be disabled with the `--no-verify` flag (two dashes) for large graphs.

4.9 Chapter Summary

This chapter has presented the five DQS-Sim scheduling algorithms:

1. MPP, unlimited ancillas (MIS + graph-aware layout + greedy interval scheduling).
2. CZ, unlimited ancillas (edge colouring, $T = 2\chi'$).
3. CZ, fixed ancilla pool (colour-class sub-batching, $T = 2 \sum_i \lceil |C_i|/k_a \rceil$).
4. MPP, switch-limited (conflict graph + bin-packing onto switches).
5. CZ, switch-limited (sub-batching by switch throughput $\ell = S \lfloor k/2 \rfloor$).

Physical resource counts use $n_{\text{phys}} = 2d^2 - 1$ per QPU. Verification bounds and implementation details ensure correctness of the simulator outputs.

Chapter 5

Verification and Theoretical Bounds

A key strength of DQS-Sim is its built-in verification layer. After each scheduling run, the simulator checks that the produced schedule satisfies known theoretical bounds. This chapter derives those bounds and describes how DQS-Sim implements and reports them.

5.1 Case 1: MPP Schedule Bounds

5.1.1 Lower Bound via Clique Number

The generator conflict graph $\mathcal{C}$ has a vertex for each generator $g_i, i \in V \setminus I$, and an edge between g_i and g_j whenever their supports overlap. In any valid schedule, all generators in a clique of $\mathcal{C}$ must be assigned to *distinct* time slots (they mutually conflict). Therefore:

$$T_1 \geq \omega(\mathcal{C}), \tag{5.1}$$

where $\omega(\mathcal{C})$ is the clique number (size of the largest clique).

5.1.2 Upper Bound via Greedy Colouring

A proper colouring of $\mathcal{C}$ assigns a time slot (colour) to each generator such that conflicting generators receive different slots. The chromatic number $\chi(\mathcal{C})$ is the minimum number of colours needed, and a greedy colouring uses at most $\Delta(\mathcal{C}) + 1$ colours, since when any generator is coloured, its conflicts can have blocked at most $\Delta(\mathcal{C})$ colours. One caveat applies: this bound is derived for the support-conflict abstraction, whereas the Case-1 scheduler enforces the stronger interval-disjointness constraint of Chapter 4, under which two support-disjoint generators can still collide. The greedy colouring value is therefore used as an empirical sanity check rather than a guaranteed ceiling,

$$\omega(\mathcal{C}) \leq T_1 \lesssim \Delta(\mathcal{C}) + 1, \tag{5.2}$$

and the check held in every instance measured in this thesis.

When is the bound tight?

On most of the families measured in this thesis (paths, stars, complete graphs, and the small grid), the measured T_1 met its clique lower bound exactly, so the greedy schedule was optimal on those instances. Where it exceeded the bound, as on cycles and large grids, the excess is the interval blocking of Chapter 4 rather than a failure of the colouring.

5.1.3 Verification Implementation

DQS-Sim computes $\omega(\mathcal{C})$ by finding the maximum clique (exact for small $|\mathcal{C}|$, approximated by greedy for large instances) and checks

$$\omega \leq T_1 \leq \Delta(\mathcal{C}) + 1. \quad (5.3)$$

An assertion failure raises a `SchedulingError` with the violated bound.

5.2 Case 2: Vizing's Theorem

5.2.1 Theoretical Statement

Theorem 5.1 (Vizing, 1964 [32]). *For any simple graph G , the chromatic index satisfies*

$$\Delta(G) \leq \chi'(G) \leq \Delta(G) + 1. \quad (5.4)$$

Since $T_2 = 2\chi'(G)$, an optimal colouring would give $2\Delta(G) \leq T_2 \leq 2(\Delta(G) + 1)$. DQS-Sim's greedy colouring can exceed the Vizing range, so the bound the simulator enforces at runtime is the greedy one,

$$2\Delta(G) \leq T_2 \leq 2(\Delta(G) + 1), \quad (5.5)$$

with any result above $2(\Delta + 1)$ flagged as suboptimal.

5.2.2 Vizing Class

Graphs are classified as:

- **Class 1:** $\chi'(G) = \Delta(G)$ (e.g., bipartite graphs, cycles C_n with even n , complete graphs K_n with even n).

- **Class 2:** $\chi'(G) = \Delta(G) + 1$ (e.g., odd cycles, the Petersen graph, complete graphs K_n with odd n).

The complete graph K_5 ($\Delta = 4$) is Vizing class 2 because $n = 5$ is odd, so its optimal chromatic index is $\chi'(K_5) = 5$. The greedy colouring is not optimal and returns 7 colours, which equals the greedy worst case of $2\Delta - 1 = 7$ and lies above the Vizing optimum of 5. This gap is the price of the fast approximation.

Example 5.2 (Cycle C_6). $\Delta(C_6) = 2$, $\chi'(C_6) = 2$ (class 1, even cycle). Tocks: $T_2 = 2 \times 2 = 4$. Verification: $2 \leq 2 \leq 3 \checkmark$.

Example 5.3 (Complete K_5). $\Delta(K_5) = 4$, optimal $\chi' = 5$ (class 2), greedy $\chi' = 7$. Tocks: $T_2 = 2 \times 7 = 14$. The greedy result exceeds the Vizing optimum $\Delta + 1 = 5$ but stays within the greedy bound $2\Delta - 1 = 7$, so the check $\Delta \leq \chi' \leq 2\Delta - 1$ gives $4 \leq 7 \leq 7 \checkmark$, with the result flagged as suboptimal.

5.2.3 Verification Implementation

DQS-Sim reports `vizing_class` as 1 or 2 (or an approximation), computed by comparing χ' to $\Delta(G)$. Because the greedy colouring is not optimal, the check enforces the greedy bound

$$\Delta(G) \leq \chi'(G) \leq 2\Delta(G) - 1, \quad (5.6)$$

rather than the tighter Vizing bound $\Delta + 1$. When the greedy result exceeds $\Delta + 1$, the simulator flags it as a suboptimal approximation of the true χ' .

5.3 Case 3: Ancilla-Constrained Monotonicity

5.3.1 Monotonicity in k_a

Proposition 5.4. $T_3(k_a)$ is non-increasing in k_a and satisfies:

$$T_3(k_a) = 2 \sum_{i=1}^{\chi'} \left\lceil \frac{|C_i|}{k_a} \right\rceil, \quad (5.7)$$

$$T_3(k_a) \geq T_2 = 2\chi' \quad \text{for all } k_a, \quad (5.8)$$

$$T_3(1) = 2|E|, \quad (5.9)$$

$$T_3(k_a) = T_2 \quad \text{when } k_a \geq \max_i |C_i|. \quad (5.10)$$

Proof. Each $\lceil |C_i|/k_a \rceil \geq 1$ with equality iff $k_a \geq |C_i|$. As k_a increases, each ceiling term decreases weakly, so $T_3(k_a)$ decreases weakly. At $k_a = 1$, $\lceil |C_i|/1 \rceil = |C_i|$, so $T_3 = 2 \sum_i |C_i| = 2|E|$. $\square$

5.3.2 Verification Check

DQS-Sim verifies that $T_3 \geq T_2$ (relaxing the ancilla constraint cannot speed up the schedule beyond the unlimited-ancilla case). This monotone decrease in k_a , from $2|E|$ at $k_a = 1$ down to the $2\chi'$ floor, is the ancilla trade-off curve shown earlier in Figure 3.1.

5.4 Switch Constraint Verification (Cases 4 and 5)

5.4.1 Feasibility Condition for MPP (Case 4)

For Case 4, the feasibility condition for scheduling generator g_i through a switch is:

$$|\text{supp}(g_i)| + 1 \leq k. \quad (5.11)$$

If the graph has a vertex of degree $\geq k-1$, its generator cannot be scheduled in a single switch round. DQS-Sim raises a warning when generators exceed the switch capacity.

In a well-designed distributed architecture, the switch port count is set to at least $\Delta(G) + 2$, so that even the highest-degree generator (support $\Delta + 1$ plus one ancilla port) remains schedulable.

5.4.2 Throughput Bound for CZ (Case 5)

The minimum number of sub-batches for colour class C_i is:

$$s_i = \left\lceil \frac{|C_i|}{S \lfloor k/2 \rfloor} \right\rceil. \quad (5.12)$$

When $S \lfloor k/2 \rfloor \geq \max_i |C_i|$, Case 5 reduces to Case 2 ($T_5 = T_2$). The switch constraint is non-binding if the switch throughput exceeds the maximum colour class size.

5.4.3 Consistency Checks

The following consistency inequalities hold by construction, because the three CZ cases share one edge colouring and differ only in how strongly the gate throughput is capped:

$$T_2 \leq T_3 \quad (\text{an ancilla cap cannot speed CZ up}), \quad (5.13)$$

$$T_2 \leq T_5 \quad (\text{a switch cap cannot speed CZ up}). \quad (5.14)$$

Cases 1 and 4 are deliberately *not* ordered. Case 4 does not inherit Case 1's one-dimensional substrate: it groups conflict-free measurements across switches with no

interval blocking, so with several switches it can undercut T_1 (Chapter 6 measures $T_4 = 2$ against $T_1 = 3$ on the eight-qubit cycle), while a single narrow switch can equally make it far slower, or infeasible outright. No inequality relates T_1 and T_2 either: which operation is faster depends on the target graph, and Chapter 6 measures the crossover between the two. The only guaranteed cross-operation statement is one-way: if the conflict clique satisfies $\omega(\mathcal{C}) \geq 2\chi'(G)$, then $T_1 \geq \omega \geq 2\chi' = T_2$ and CZ preparation cannot lose.

5.5 Bound Summary

Table 5.1 summarises all theoretical bounds verified by DQS-Sim.

Case	Lower bound	Upper bound
1	$\omega(\mathcal{C})$	$\Delta(\mathcal{C}) + 1$
2	$2\Delta(G)$	$2(2\Delta(G) - 1)$ (greedy)
3	$T_2 = 2\chi'$	$2 E $ (at $k_a = 1$)
4	$\omega(\mathcal{C})$	measured (no closed form)
5	$T_2 = 2\chi'$	$2\lceil E /\ell \rceil \cdot \chi'$

Table 5.1: Bounds for each DQS-Sim case as enforced at runtime. Case 4 has no closed-form upper bound, so its schedule depth is reported as measured.

5.6 Chapter Summary

This chapter has derived and stated:

- The clique-number lower bound and greedy colouring upper bound for Case 1.
- Vizing’s theorem for Case 2 and the Vizing class classification.
- Monotonicity of $T_3(k_a)$ and its limiting cases.
- Feasibility and throughput conditions for Cases 4 and 5.
- Cross-case consistency inequalities.

All bounds are automatically checked by DQS-Sim, providing confidence that the scheduling outputs are correct and consistent with theory.

Chapter 6

Results

This chapter presents DQS-Sim’s measurements and turns them into design guidance. It asks one central question. Given a target graph, should it be prepared with MPP measurements or with CZ gates? The results will show that neither operation always wins. Two computable properties of the graph decide the faster choice, and the hardware constraints shift that boundary in measurable ways.

The chapter proceeds in five stages. Two worked examples, the sparse cycle C_6 and the dense complete graph K_5 , first walk through all five scheduling cases in full detail, down to which qubits are measured in which Tock and which switch ports they occupy. The MPP versus CZ section then compares the two operations head to head. A survey across six graph families shows MPP winning throughout, two further families show CZ overtaking it, first through measurement conflicts and then through routing congestion, and a closing chart shows the winner on every tested graph. A scaling study measures how the costs grow with graph size, a hardware study sweeps the ancilla pool and the switch port count to extract sizing rules, and a final section collects the evidence that the reported numbers can be trusted.

6.1 Experimental Setup

All results are produced by DQS-Sim at surface code distance $d = 7$, so each QPU comprises $2d^2 - 1 = 97$ physical qubits. Unless stated otherwise, the ancilla-limited Case 3 uses a pool of k_a ancilla QPUs given per experiment, and the switch-limited Cases 4 and 5 route every operation through S reconfigurable switches of k ports each. Both switch parameters are user inputs to the simulator; the default configuration here is a single four-port switch ($k = 4$, $S = 1$), and Section 6.6.1 varies the switch count. Five figure types recur throughout. A *schedule heatmap* plots qubits against Tocks and colours each cell by the qubit’s role in that Tock, so the reader can follow the entire schedule at a glance. A *layout drawing* shows the same schedule on the graph itself, colouring vertices and edges by their time step. A *substrate occupancy view* unfolds

the Case-1 schedule Tock by Tock, showing which data and ancilla positions each measurement engages and blocks. A *port-wiring diagram* shows, for one time step, which QPUs plug into which switch ports. A *port-utilisation chart* aggregates that information, showing how many ports each scheduled group occupies. Throughout the chapter each case keeps a fixed colour: blue for Case 1, orange for Case 2, green for Case 3, purple for Case 4, and light blue for Case 5.

6.2 Worked Example: Cycle Graph C_6

The cycle C_6 has $n = 6$ vertices and $m = 6$ edges. It is 2-regular ($\Delta = 2$) and bipartite, so its edges can be coloured with only Δ colours [33], and it has a maximum independent set of size $\alpha(C_6) = 3$, shown in Figure 6.1.

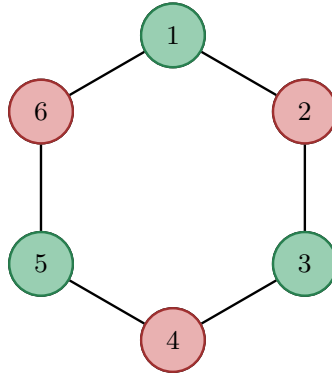


Figure 6.1: Cycle graph C_6 , with the maximum independent set in green and the vertex cover in red.

6.2.1 Cases 1–3: Operation Type and Ancilla Pool

Case 1 (MPP, unlimited ancillas). In Case 1 the maximum independent set $\{1, 3, 5\}$ is initialised in $|+\rangle$ at no measurement cost, leaving the three vertex-cover generators

$$g_2 = Z_1 X_2 Z_3, \quad g_4 = Z_3 X_4 Z_5, \quad g_6 = Z_5 X_6 Z_1$$

to be measured. Each pair of these generators shares a qubit, so their conflict graph is a triangle with clique number $\omega(\mathcal{C}) = 3$, and the three measurements occupy three separate Tocks: $T_1 = 3$, with one ancilla QPU and $(6 + 1) \times 97 = 679$ physical qubits.

The Tock count is governed by the vertex-to-qubit mapping of Chapter 4. The layout step assigns the six ring vertices to substrate positions, and under this mapping the three measurements occupy the ancilla intervals $[0, 2]$, $[1, 5]$, and $[2, 5]$. Figure 6.2

shows the resulting occupancy Tock by Tock. In Tock 2 the measured generator touches the data qubits at positions 1, 3, and 5, and its ancilla patch stretches across positions 1 through 5, so the ancillas above the idle positions 2 and 4 are blocked as well. This is the substrate's central scheduling constraint: an ancilla region occupies every position it passes over, so two measurements can collide through their routing intervals even when their supports are disjoint, and here the three intervals overlap pairwise, forcing the fully sequential schedule.

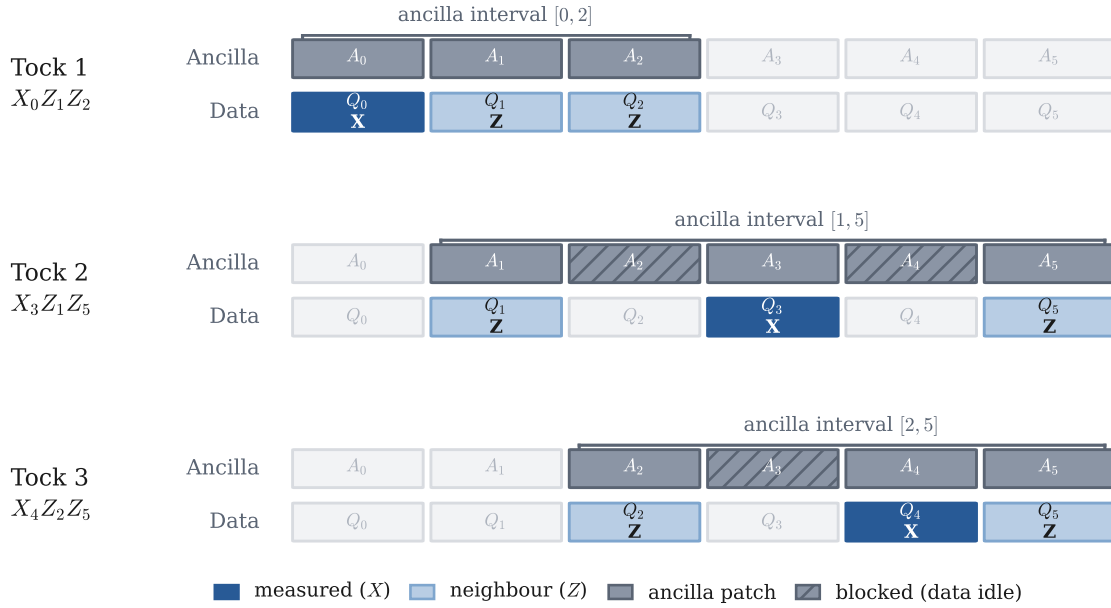


Figure 6.2: Substrate occupancy for C_6 under Case 1 (MPP), one row of strips per Tock. The measured qubit is dark blue, its neighbours light blue, the ancilla patch grey, and hatched cells are blocked positions.

Case 2 (CZ, unlimited ancillas). In Case 2, C_6 is class 1 with $\chi'(C_6) = \Delta = 2$, and the two colour classes $\{(1, 2), (3, 4), (5, 6)\}$ and $\{(2, 3), (4, 5), (6, 1)\}$ are perfect matchings of three edges. With unlimited ancillas each class executes in one round of two Tocks, giving $T_2 = 2\chi' = 4$, a peak ancilla demand of three, and $(6 + 3) \times 97 = 873$ physical qubits.

Case 3 (CZ, one ancilla). In Case 3 the pool is restricted to $k_a = 1$, so every CZ gate runs alone and each colour class splits into three single-gate sub-batches, giving $T_3 = 2(3 + 3) = 12$ Tocks. This is three times slower than Case 2 yet uses the same 679 physical qubits as Case 1, since the lone ancilla serialises the gates. The full schedules of all three cases appear in Figure 6.3, and Figure 6.4 draws the same three schedules

on the graph itself, where each colour is one time step. A Case-1 Tock lasts one Tock, while every CZ round and every CZ batch lasts two.

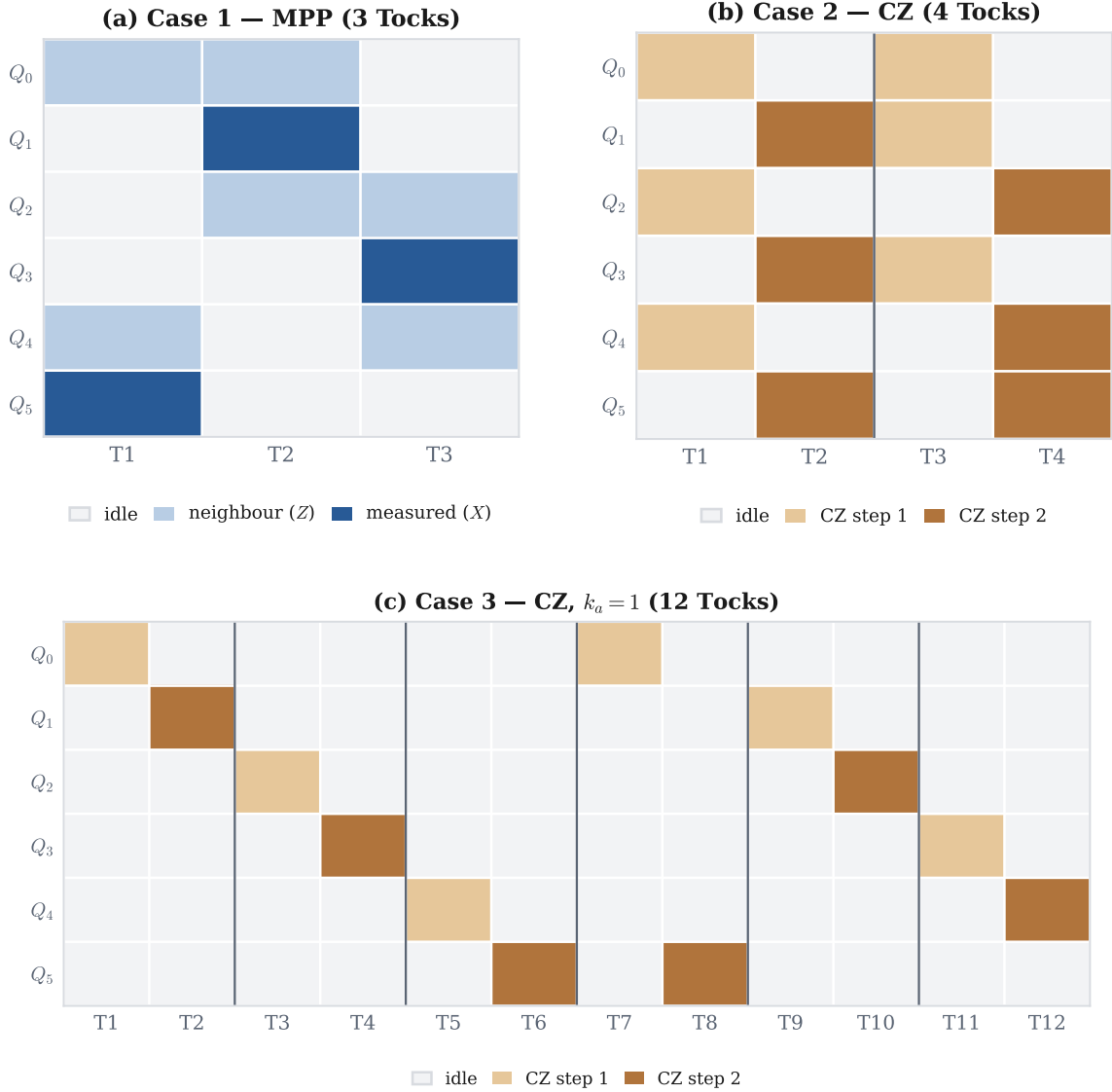


Figure 6.3: Schedule heatmaps for C_6 under Cases 1 to 3. Blues mark the MPP measurements (Case 1) and oranges the two steps of each CZ gate (Cases 2 and 3).

6.2.2 Cases 4–5: Switch Connectivity

Case 4 (MPP, switch-limited). Routing the MPP schedule through the four-port switch costs nothing for C_6 . Each generator has support over three qubits and so needs $3 + 1 = 4$ ports, exactly the switch width, and the three mutually conflicting measurements again occupy one group each: $T_4 = 3 = T_1$.

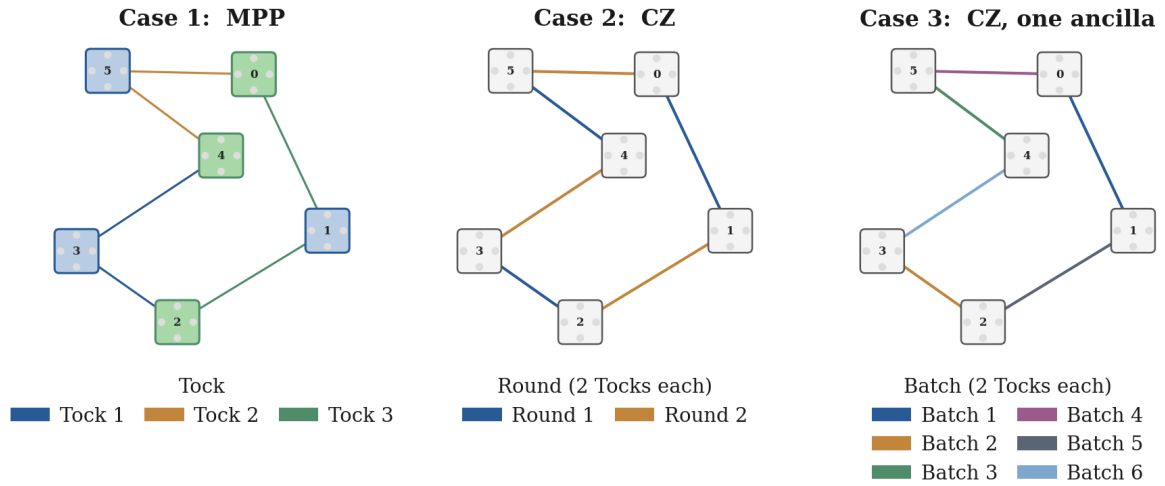


Figure 6.4: The C_6 schedules drawn on the graph. Case 1 (MPP) colours each measured vertex and its edges by measurement Tock, with the independent-set vertices in green. Cases 2 and 3 (CZ) colour each edge by its round or batch.

Case 5 (CZ, switch-limited). The CZ schedule is instead throttled by switch *throughput*. The switch carries only $\lfloor k/2 \rfloor = 2$ CZ gates at a time, so each three-edge colour class splits into two sub-batches and

$$T_5 = 2 \sum_{i=1}^2 \left\lceil \frac{3}{2} \right\rceil = 8 \text{ Tocks}, \quad (6.1)$$

twice the unlimited Case 2. Figure 6.6 shows both switch-limited schedules as heatmaps. In Case 4 every measurement wires the three support QPUs and the ancilla QPU through the four ports, consuming the entire switch for its group. Figure 6.7 aggregates the resulting port occupancy: Case 4 runs at 100% utilisation in every group, while Case 5 alternates between full and half occupancy (75% on average) as the odd-sized colour classes wrap. Figure 6.5 traces the wiring itself, step by step. Each MPP Tock plugs the three support QPUs and the ancilla into all four ports, and each CZ round plugs in at most two gates of two ports each. The CZ ancillas sit beside their data QPUs on fixed local links, so they never occupy switch ports.

Because Case 5's bottleneck is throughput, it can be bought back by increasing the switch count: with two four-port switches ($S = 2$) the capacity rises to $S \lfloor k/2 \rfloor = 4$ simultaneous CZ gates, each colour class again fits in a single sub-batch, and the schedule returns to the unlimited floor $T_5 = 4 = T_2$. The MPP schedule gains nothing from the second switch here ($T_4 = 3$ either way), since its three measurements conflict and can never run in parallel; Section 6.6.1 shows a graph where the second switch does halve the MPP schedule.

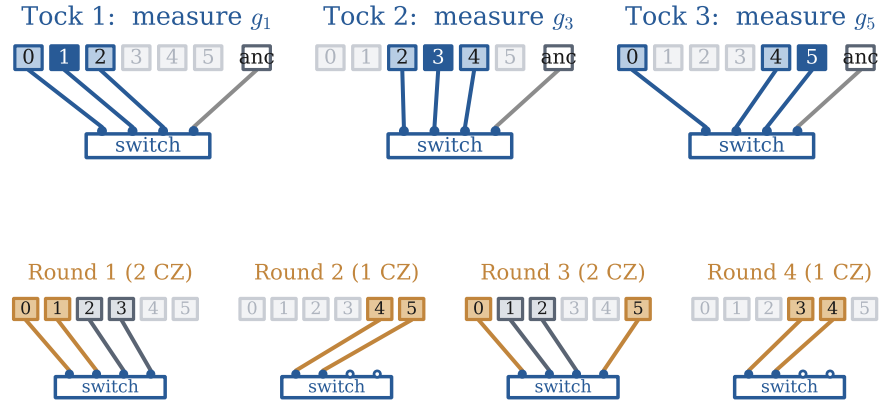


Figure 6.5: Port wiring for C_6 on one four-port switch. Top row shows Case 4 (MPP) and bottom row Case 5 (CZ).

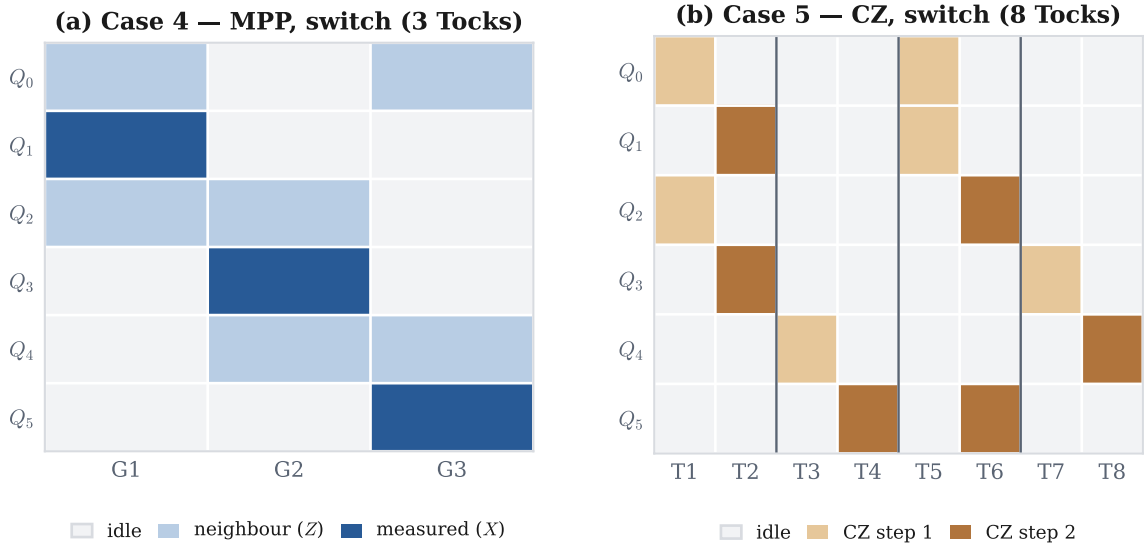


Figure 6.6: Schedule heatmaps for C_6 on one four-port switch, Case 4 (MPP) left and Case 5 (CZ) right.

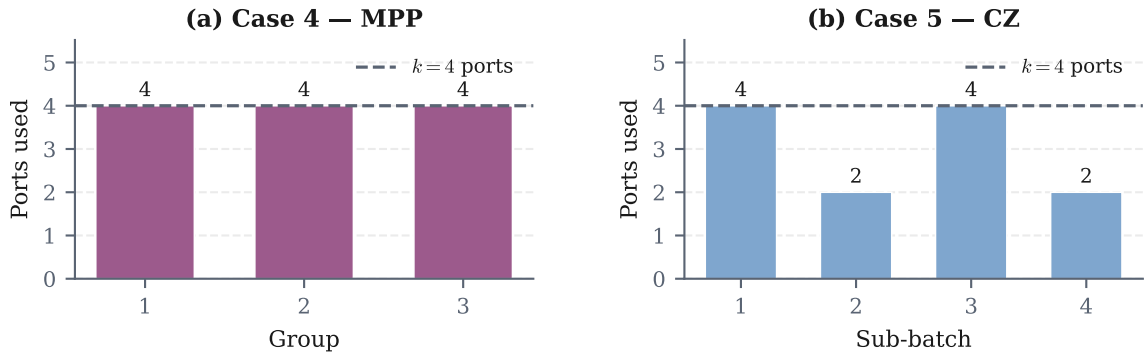


Figure 6.7: Switch port utilisation for C_6 on one four-port switch, Case 4 (MPP) left and Case 5 (CZ) right.

6.3 Worked Example: Complete Graph K_5

K_5 has $n = 5$ vertices, $m = 10$ edges, and is 4-regular ($\Delta = 4$). Like every odd complete graph it needs one colour more than its maximum degree, so $\chi'(K_5) = 5$ theoretically. The MIS has size $\alpha(K_5) = 1$, any single vertex, as Figure 6.8 shows.

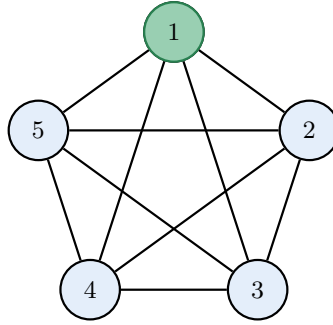


Figure 6.8: Complete graph K_5 , with the single-vertex maximum independent set in green and the vertex cover in red.

6.3.1 Cases 1–3: Operation Type and Ancilla Pool

Case 1 (MPP, unlimited ancillas). In Case 1 a single vertex joins the MIS and the remaining four each require a generator measurement. Because the graph is complete, every generator has support over the entire vertex set, so all four mutually conflict ($\omega(\mathcal{C}) = 4$) and $T_1 = 4$ Tocks, with one ancilla and $(5 + 1) \times 97 = 582$ physical qubits.

Cases 2 and 3 (CZ). In Case 2 the greedy edge colouring returns $\chi' = 7$ colour classes, above the Vizing optimum of five for an odd complete graph, with class sizes of one or two. This yields $T_2 = 14$ Tocks, a peak ancilla demand of two, and 679 physical qubits. The MPP schedule is a factor $T_2/T_1 = 3.5$ faster, the largest advantage in these examples. In Case 3 with $k_a = 2$ every colour class already fits in a single sub-batch, so the ancilla limit is not binding and $T_3 = T_2 = 14$. Figure 6.9 contrasts the four-Tock MPP schedule with the fourteen-Tock CZ schedules, and Figure 6.10 draws all three on the graph.

6.3.2 Cases 4–5: Switch Connectivity

Case 4 (MPP, switch-limited). K_5 exposes the switch *width* constraint. Every generator spans all five vertices, so an MPP measurement needs $5 + 1 = 6$ ports and cannot be hosted by the four-port switch at all: DQS-Sim reports Case 4 as infeasible

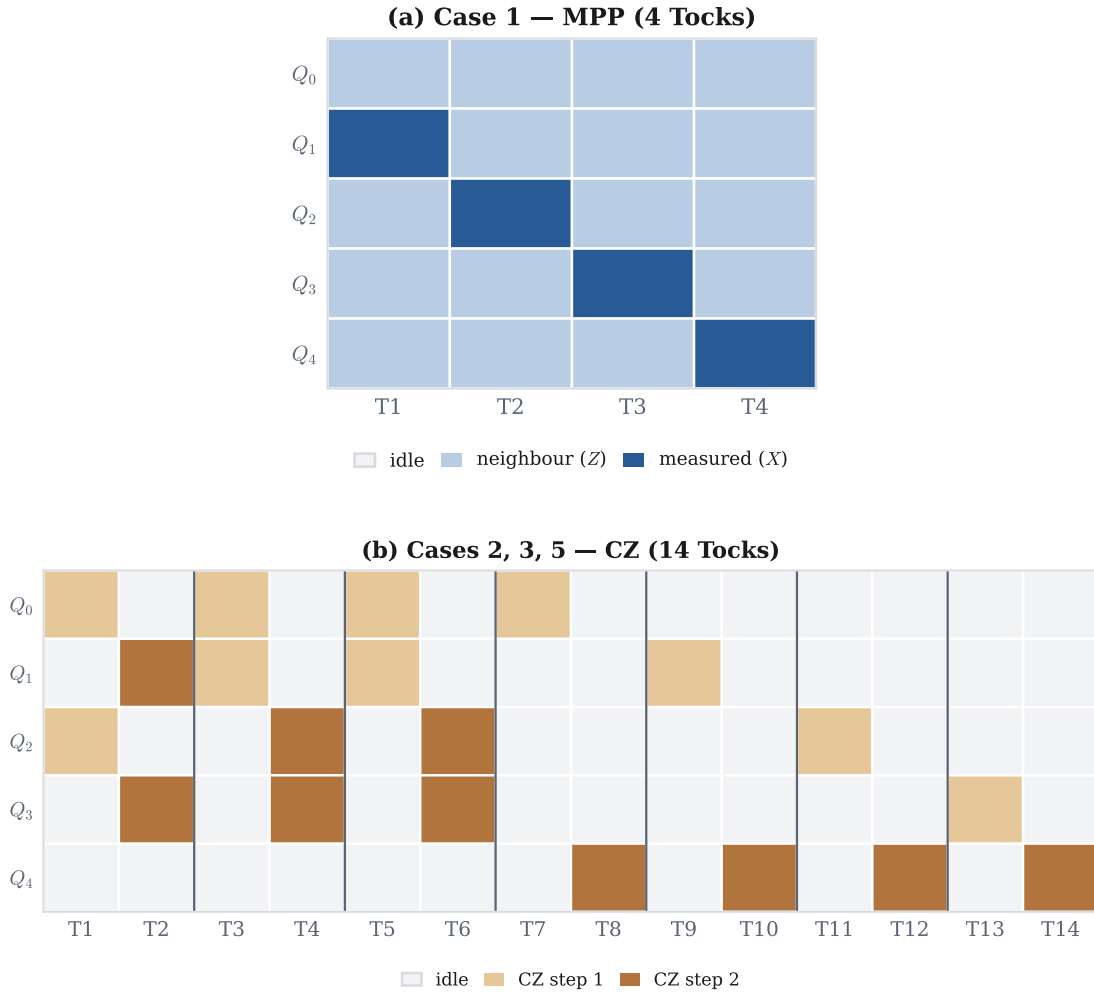


Figure 6.9: Schedule heatmaps for K_5 , Case 1 (MPP) on top and the CZ schedule below, which Cases 2, 3, and 5 share on this graph.

and requires $k \geq 6$, at which point the schedule completes in $T_4 = 4$ groups, matching Case 1. Adding switches does not help, because all qubits of one measurement must connect through the same switch, so a second four-port switch leaves Case 4 exactly as infeasible ($S = 2$, $k = 4$ was verified directly) and only a wider switch admits the measurement.

Case 5 (CZ, switch-limited). CZ preparation is far less demanding, since each gate occupies only two ports. The four-port switch carries two CZ gates at a time, which matches the largest colour class, so every class fits in one sub-batch and $T_5 = 14 = T_2$: the switch is no bottleneck for CZ on this graph (Figure 6.11). Figure 6.12 shows both situations at the port level, the six-port measurement that cannot be wired and a two-gate CZ round that fits with room to spare.

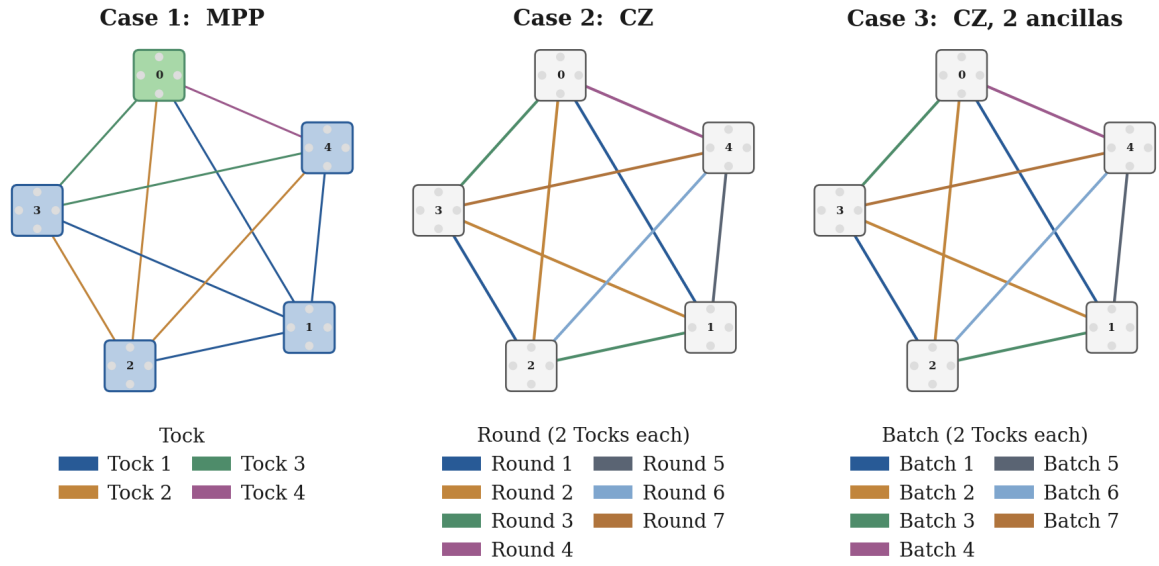


Figure 6.10: The K_5 schedules drawn on the graph. Case 1 (MPP) colours each measured vertex and its edges by measurement Tock, with the single independent-set vertex in green. Cases 2 and 3 (CZ) colour each edge by its round or batch.

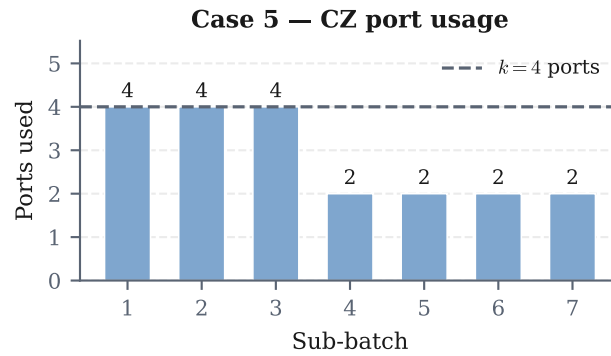


Figure 6.11: Switch port utilisation for K_5 on one four-port switch. Only Case 5 (CZ) appears, since Case 4 (MPP) is infeasible here.

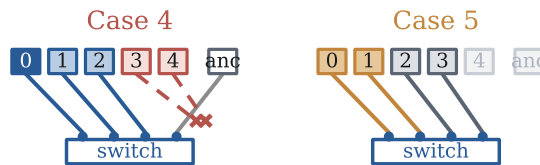


Figure 6.12: Port wiring for K_5 on one four-port switch. Left shows the infeasible Case 4 (MPP) measurement and right one Case 5 (CZ) round.

6.3.3 Summary of the Worked Examples

Table 6.1 collects the numbers. The two graphs bracket the design space: on the sparse cycle the switch throughput throttles CZ while MPP is untouched, and on the dense complete graph the switch width excludes MPP entirely while CZ is untouched.

Graph	Case	Tocks	Peak anc. QPUs	N_{phys}	T_c/T_1
C_6	1 (MPP, ∞)	3	1	679	1.00
C_6	2 (CZ, ∞)	4	3	873	1.33
C_6	3 (CZ, $k_a = 1$)	12	1	679	4.00
C_6	4 (MPP, switch)	3	1	679	1.00
C_6	5 (CZ, switch)	8	2	776	2.67
K_5	1 (MPP, ∞)	4	1	582	1.00
K_5	2 (CZ, ∞)	14	2	679	3.50
K_5	3 (CZ, $k_a = 2$)	14	2	679	3.50
K_5	4 (MPP, switch)	infeasible at $k = 4$ (needs $k \geq 6$)			
K_5	5 (CZ, switch)	14	2	679	3.50

Table 6.1: DQS-Sim results for the worked examples at code distance $d = 7$. Cases 4 and 5 use a single $k = 4$ -port switch ($S = 1$). T_c/T_1 is the Tock ratio relative to Case 1.

6.4 MPP versus CZ

The central design question DQS-Sim answers is which entangling operation to prepare with, and the measured answer is that neither operation dominates. Two properties of the target graph decide the winner. The first is *conflict density*: MPP measurements whose supports overlap must serialise, so MPP depth is driven by how many generators mutually conflict. The second is *layout congestion*: on the one-dimensional substrate each measurement’s ancilla blocks its whole routing interval, so MPP depth also grows when the graph cannot be laid on a line without long edges. CZ preparation is immune to both, paying only the degree through $T_2 = 2\chi'(G)$. This section measures each effect in isolation: a fixed-size survey where neither effect appears and MPP leads throughout, a graph family engineered to flip the order through conflicts alone, a second family that flips through congestion alone, and a closing chart showing which operation wins where.

6.4.1 Survey at Fixed Size

To establish the baseline beyond two hand-picked graphs, DQS-Sim was run on six graph families of comparable size, spanning sparse chains to dense cliques and including

the 3×3 grid, whose graph state is the canonical cluster-state resource of measurement-based computing. All five cases use the uniform parameters $k_a = 2$ and a single four-port switch. Table 6.2 and Figure 6.13 report the schedule depths.

Graph	n	m	Δ	χ'_{greedy}	T_1	T_2	T_3	T_4	T_5
Path P_8	8	7	2	2	2	4	8	4	8
Cycle C_8	8	8	2	2	3	4	8	4	8
Star $K_{1,7}$	8	7	7	7	1	14	14	$\times$	14
Complete K_8	8	28	7	7	7	14	28	$\times$	28
Grid 3×3	9	12	4	4	4	8	14	$\times$	14
Random $G(8, 0.35)$	8	15	5	6	5	12	18	$\times$	18

Table 6.2: Schedule depth in Tocks for six graph families under all five cases ($k_a = 2$ ancillas for Case 3; one $k = 4$ -port switch for Cases 4 and 5). $\times$ marks Case-4 instances that are infeasible because a generator needs $\Delta + 2 > 4$ ports.

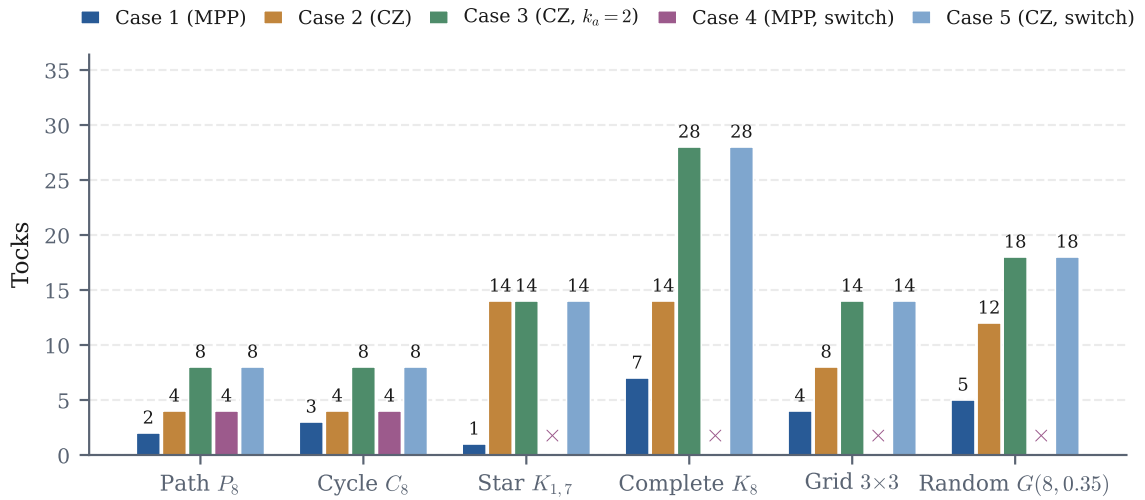


Figure 6.13: Schedule depth by graph family and case ($k_a = 2$, one four-port switch). Bars are labelled with the Tock count and $\times$ marks infeasible Case-4 instances.

Four observations hold across the survey. First, the unlimited MPP schedule is the shallowest on every family, from a modest 25% saving on the path to a factor of fourteen on the star. This is a property of the families and the size chosen rather than of MPP itself: in all six graphs, the number of mutually conflicting generators never outgrows the degree, and at eight or nine vertices the layout congestion is negligible, so neither of the two effects that penalise MPP has room to act. Sections 6.4.2 and 6.4.3 let each effect act in turn.

Second, the Case-4 column shows how strongly the switch width selects against MPP: with four ports only the degree-2 families survive, because a generator of degree

Δ needs $\Delta+2$ ports. Third, the Case-3 and Case-5 columns coincide on every row. This is the identity predicted in Chapter 4: a switch carrying $\lfloor k/2 \rfloor$ CZ gates behaves exactly like an ancilla pool of that size, and with $k_a = 2 = \lfloor 4/2 \rfloor$ the two schedules agree, which serves as a built-in consistency check between two independently implemented schedulers.

Fourth, the grid, the structurally closest family to practical MBQC cluster states, behaves like a moderate-density graph at this size: MPP halves the CZ schedule, but its degree-4 generators already demand six-port switches, and Section 6.4.3 shows the depth advantage itself erodes as the grid grows.

6.4.2 The Conflict Flip

The first way MPP loses follows from a bound. Two measurements that touch a common qubit cannot share a Tock, so a group of measurements that all pairwise touch common qubits must run one per Tock. The MPP depth is therefore at least the size of the largest such group, the conflict clique, giving $T_1 \geq \omega(\mathcal{C})$. The CZ schedule costs exactly $T_2 = 2\chi'(G)$, which Vizing's theorem ties to the maximum degree. Comparing the two gives a one-way guarantee:

$$\omega(\mathcal{C}) \geq 2\chi'(G) \implies T_1 \geq T_2, \quad (6.2)$$

so whenever the conflict clique reaches twice the chromatic index, CZ preparation cannot lose. The survey families never reach that point, for one of two reasons. In the dense families, the complete graph and the star, any conflicts come from vertices of very high degree, and a high degree drives the CZ depth up just as fast, so the comparison never tips. In the sparse families, the path, the cycle, and the grid, each measurement touches only its immediate neighbourhood, so only small groups of measurements ever block one another. This is why MPP wins throughout Table 6.2. Flipping the comparison needs the one structure the survey lacks, many *low-degree* vertices that pairwise share a neighbour, so that conflicts pile up while the CZ depth stays low.

Finite geometry provides exactly that structure. In the incidence graph of a projective plane $\text{PG}(2, q)$, built from points and lines as in Figure 6.14, every two points lie on a common line. All the point measurements therefore block one another, while the degree stays at only $q + 1 \approx \sqrt{n/2}$. Table 6.3 reports the measured results for the four smallest planes alongside the six survey families, and Figure 6.15(a) plots the incidence family. The smallest member of the family, $\text{PG}(2, 2)$, is the Heawood graph shown in Figure 6.14, and it sits on the boundary: its seven point generators are mutually in conflict, forcing $T_1 = \omega = 7$ against $T_2 = 8$, so MPP survives by a single Tock. One

plane later the order inverts, with $\text{PG}(2, 3)$ already meeting the rule at $\omega = 10 = 2\chi'$, and by $\text{PG}(2, 5)$ the CZ schedule is shallower by a factor of 2.5 that grows as $\Theta(\sqrt{n})$ without bound: the mirror image of the star family, where the MPP advantage grows as n .

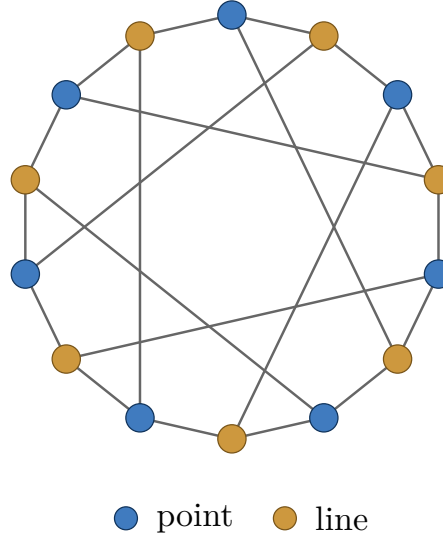


Figure 6.14: The Heawood graph, the smallest incidence graph in the family, $\text{PG}(2, 2)$. Blue vertices are its seven points and gold its seven lines, with an edge whenever a point lies on a line.

Graph	n	Δ	χ'	$\omega(\mathcal{C})$	T_1	T_2	Faster
Path P_8	8	2	2	2	2	4	MPP
Cycle C_8	8	2	2	2	3	4	MPP
Star $K_{1,7}$	8	7	7	1	1	14	MPP
Complete K_8	8	7	7	7	7	14	MPP
Grid 3×3	9	4	4	4	4	8	MPP
Random $G(8, 0.35)$	8	5	6	5	5	12	MPP
$\text{PG}(2, 2)$ (Heawood)	14	3	4	7	7	8	MPP
$\text{PG}(2, 3)$	26	4	5	10	18	10	CZ
$\text{PG}(2, 4)$	42	5	7	18	31	14	CZ
$\text{PG}(2, 5)$	62	6	8	29	40	16	CZ

Table 6.3: The conflict flip. $\omega(\mathcal{C})$ is the exact clique number of the generator conflict graph and χ' the greedy chromatic index, so $T_2 = 2\chi'$. Comparing ω against $2\chi'$ identifies the faster operation on all ten graphs.

6.4.3 The Congestion Flip

The second way MPP loses has nothing to do with conflicts. Table 6.4 scales the square grid, the cluster-state family, from 2×2 to 7×7 . The conflict clique stays fixed at $\omega = 4$, so the rule of Equation (6.2) never predicts a CZ win, yet the measured T_1 climbs steadily and crosses the constant $T_2 = 8$ at the 6×6 grid (Figure 6.15(b)).

Grid	n	χ'	$\omega(\mathcal{C})$	T_1	T_2	Faster
2×2	4	2	2	2	4	MPP
3×3	9	4	4	4	8	MPP
4×4	16	4	4	5	8	MPP
5×5	25	4	4	7	8	MPP
6×6	36	4	4	9	8	CZ
7×7	49	4	4	10	8	CZ

Table 6.4: The congestion flip on square grids. The conflict clique stays flat at $\omega = 4$, yet T_1 grows with the grid side and overtakes the constant $T_2 = 8$ at 6×6 .

The cause is the substrate’s interval-blocking rule. However the grid’s vertices are arranged along a line, roughly $\sqrt{n}$ of its edges must stretch across the middle, an unavoidable overlap known as the graph’s *cutwidth*. Every stretched edge gives some measurement a long ancilla interval, so about $\sqrt{n}$ measurements pile up over the middle of the substrate and take turns. The measured depths follow this pattern, with T_1 rising by roughly one Tock per unit of grid side while ω stays flat, so the entire loss is congestion, not conflicts. Unlike the conflict flip, this effect is a property of the one-dimensional substrate model inherited from the monolithic Substrate Scheduler rather than of MPP itself; a two-dimensional routing model would be expected to recover most of it. Within the model, however, the consequence is concrete: for large cluster states on a linear-bus architecture, CZ preparation is the faster choice.

6.4.4 Which Operation Wins

Figure 6.15(c) places every tested graph by its conflict ratio $\omega(\mathcal{C})/2\chi'$, the quantity the rule checks, against the depth ratio T_1/T_2 the simulator actually measured. The shading names the winner directly. Every graph in the gold upper half prepared faster with CZ, and every graph in the blue lower half prepared faster with MPP. The blue circles, the common families, all stay in the MPP half. The gold circles, the incidence family, climb into the CZ half once past the dashed threshold where the rule starts predicting CZ, with the smallest plane sitting just below the line as the boundary case. The red triangles are the large grids. Two of them sit in the CZ half even though

they are far left of the threshold, and that gap between prediction and outcome is the congestion effect.

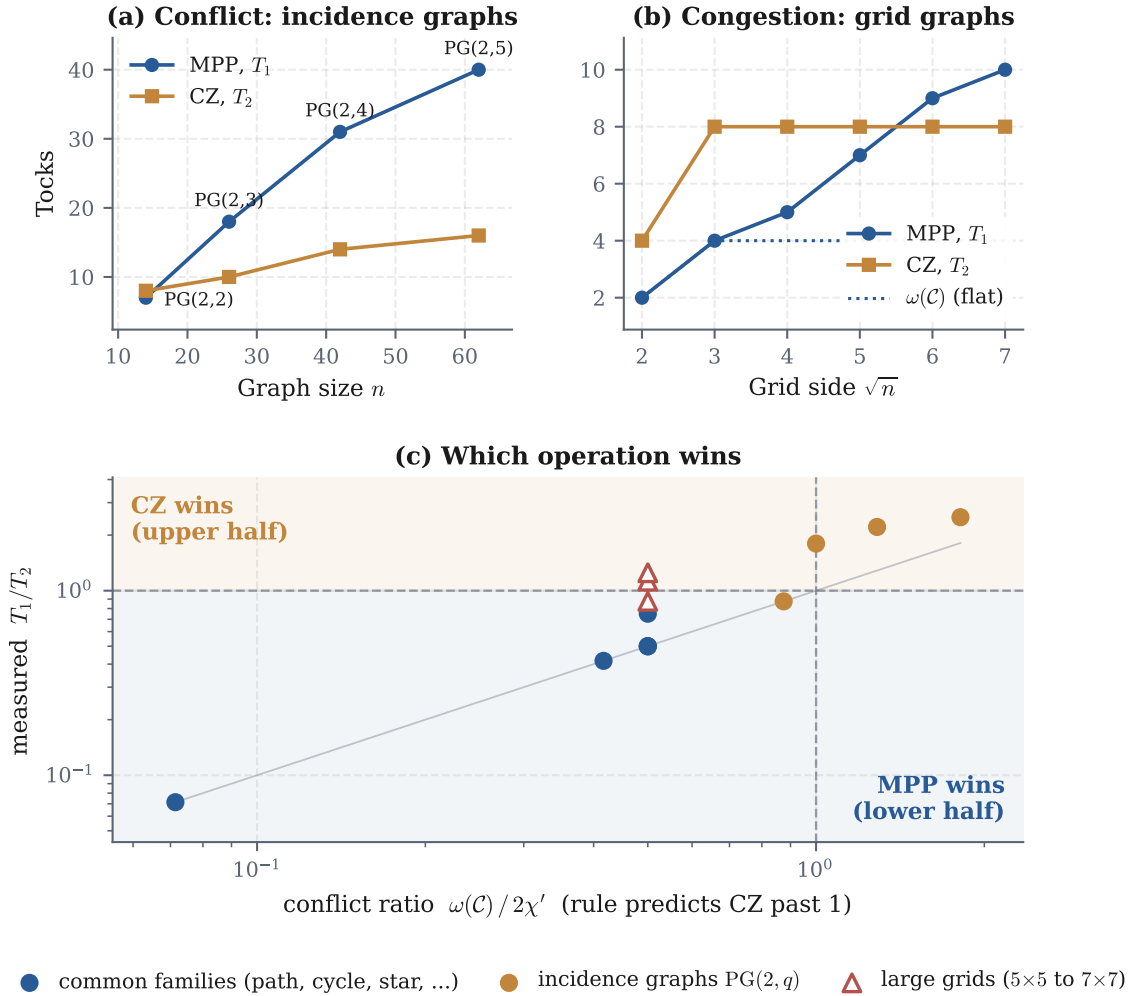


Figure 6.15: MPP versus CZ. (a) Schedule depth on the incidence family. (b) Schedule depth on square grids, with the flat conflict floor dotted. (c) Every tested graph placed by conflict ratio against measured depth ratio. The gold upper half holds the CZ wins, the blue lower half the MPP wins, and the red triangles are the large grids.

The chart reads as a two-step rule for a compiler, using quantities DQS-Sim already computes. First compare conflicts. If $\omega(C) \geq 2\chi'(G)$, prepare with CZ, since Equation (6.2) then guarantees the win. Otherwise MPP is the likely winner, and the remaining question is congestion. If the graph lays out comfortably on a line, as paths, cycles, stars, and cliques all do, the measured MPP schedule stays at or near its conflict floor, and MPP won at every size we tested. If it does not, as for two-dimensional grids, the congestion penalty grows with size and hands large instances to CZ. Two caveats bound the rule. The reported χ' is the greedy value, so an optimal colouring would shift the boundary further in CZ's favour, and the congestion term is a property of

the one-dimensional substrate model, so architectures with richer routing would shift it back toward MPP.

6.5 Scaling with Graph Size

Figure 6.16 measures how the two unlimited cases grow with the number of qubits, sweeping $n = 4$ to 20 for four families; the grid family’s scaling was covered by Table 6.4.

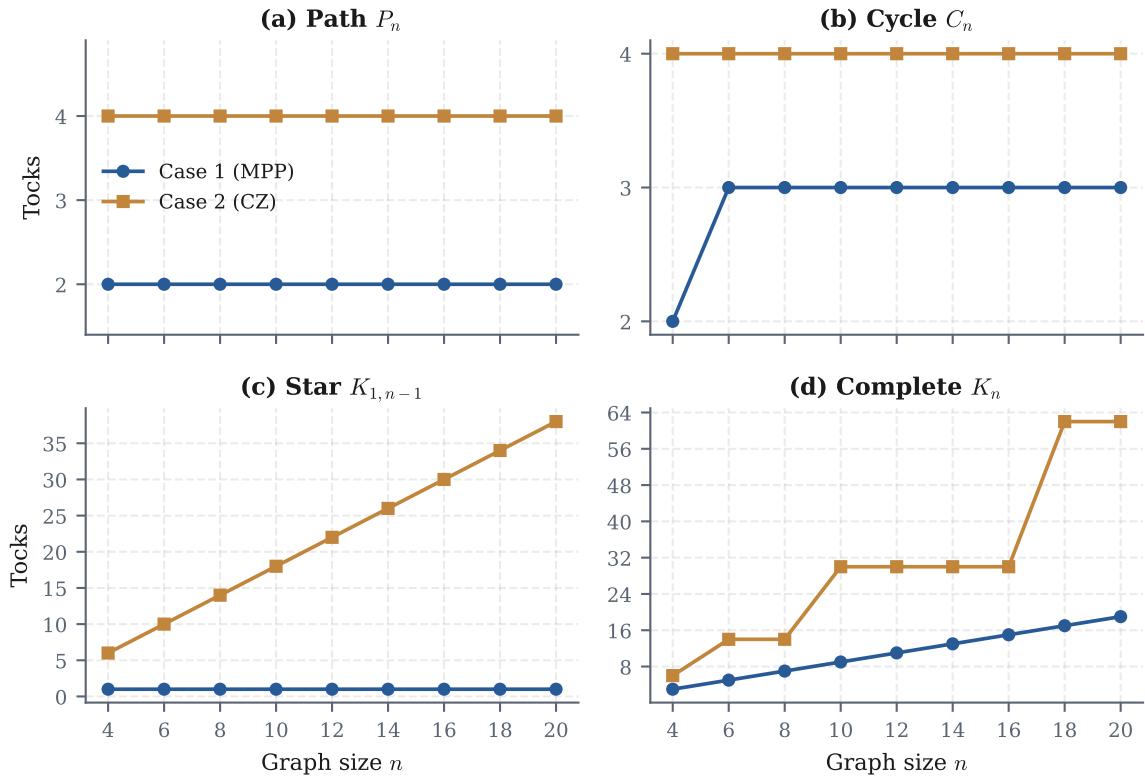


Figure 6.16: Measured schedule depth versus graph size for Case 1 (MPP, blue) and Case 2 (CZ, orange) on four graph families.

For the sparse families the measured depths are flat: paths cost $T_1 = 2$ and cycles $T_1 = 3$ Tocks at every size beyond the smallest. On a path the layout step lays the chain out in order, so alternating generators have disjoint ancilla intervals and pack into two rounds regardless of length. A cycle needs one Tock more because it cannot be laid on a line without one wrap-around generator whose ancilla interval spans the whole substrate, blocking everything else for one Tock, a direct consequence of the interval-blocking rule of Chapter 4. Meanwhile $T_2 = 2\chi' = 4$ stays constant since $\chi' = 2$ for both families. The constant survives far beyond the sweep: a thousand-qubit cycle completes in four Tocks under both cases (using the greedy MIS heuristic that replaces the exact solver beyond $n = 20$), so for such states only the QPU count

grows with size, not the schedule.

The star gives the opposite extreme. Its single hub generator gives $T_1 = 1$ at every size, while every edge shares the hub, forcing $\chi' = n - 1$ and a linearly growing $T_2 = 2(n - 1)$: the MPP advantage is unbounded in n . On complete graphs both cases grow linearly, with MPP holding a consistent factor of two or more ($T_1 = n - 1$ exactly). The CZ curve additionally shows a staircase, jumping at $n = 10$ and $n = 18$; these are not physics but the greedy edge colouring falling further behind the Vizing optimum of $\approx n$ colours, an algorithmic overhead that DQS-Sim flags and that an optimal colouring subroutine would remove.

6.6 Hardware Sizing: Ancillas, Ports, and Switch Count

The final experiment asks the design question directly: how many ancilla QPUs, and how many switch ports, does a target graph actually need? Figure 6.17 sweeps both parameters for a sparse instance (C_{12}) and a dense one (K_8).

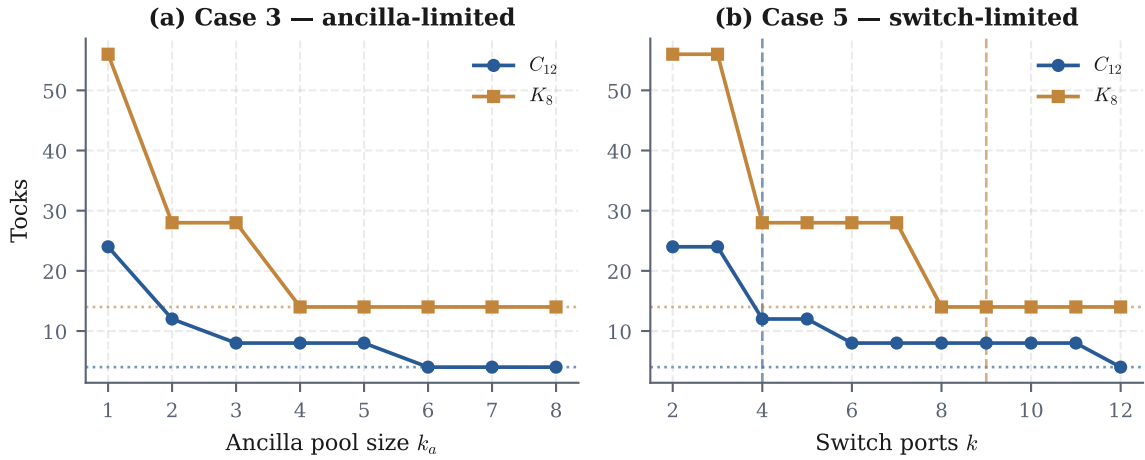


Figure 6.17: Measured hardware trade-off curves for C_{12} (blue) and K_8 (orange). (a) Case-3 schedule depth versus ancilla pool size k_a ; dotted lines mark each graph's unlimited-ancilla floor $T_2 = 2\chi'$. (b) Case-5 depth versus switch port count k ; dashed vertical lines mark the smallest switch on which Case 4 (MPP) becomes feasible, at $k = \Delta + 2$.

Panel (a) confirms the ancilla trade-off anticipated in Chapter 3 with measured data. The depth follows the staircase $T_3 = 2\sum_i \lceil |C_i|/k_a \rceil$, falling from $2|E|$ at $k_a = 1$ (24 and 56 Tocks) and flattening onto the floor T_2 exactly when the pool reaches the largest colour class, at $k_a = 6$ for C_{12} and $k_a = 4$ for K_8 , and ancillas beyond that point add nothing. Panel (b) shows the same staircase for the switch-limited CZ schedule

with the effective pool $\lfloor k/2 \rfloor$, reaching the floor at $k = 12$ and $k = 8$ respectively. The dashed thresholds summarise the MPP side: Case 4 is infeasible left of $k = \Delta + 2$ (four ports for the cycle, nine for K_8) and needs no more than that to match its unlimited depth on these graphs.

Together the two panels give three quantitative sizing rules for a target graph G . CZ-based preparation needs $k_a \geq \max_i |C_i|$ ancillas, or equivalently $\lfloor k/2 \rfloor \geq \max_i |C_i|$ switch ports, to run at full speed, and below that threshold halving the pool roughly doubles the depth. MPP-based preparation needs $k \geq \Delta(G) + 2$ ports before it runs at all, and then delivers the shallower schedule whenever the crossover criterion of Section 6.4 favours it. A hardware designer can therefore read the required switch generation directly from the degree and colour-class profile of the target graph family.

6.6.1 Scaling the Switch Count

The switch count S is the simulator’s second connectivity input, and sweeping it shows what more switches can and cannot buy. Figure 6.18 shows the mechanism at its best: on the eight-qubit cycle, the Case-4 generators are not all in conflict, so with two four-port switches two measurements run in the same Tock, one wired through each switch, and the schedule halves from $T_4 = 4$ to $T_4 = 2$. This even undercuts the monolithic baseline $T_1 = 3$: switched routing has no shared substrate bus for ancilla intervals to jam on, so once enough switches exist, Case 4 escapes the congestion effect of Section 6.4.3 that Case 1 can never shed. The same parallelism drives the CZ side. Figure 6.19 shows one round of the same cycle’s CZ schedule on two switches. The colour class’s four gates run two per switch in a single round, so T_5 falls from eight Tocks on one switch to its unlimited floor of four. Figure 6.20 sweeps S at fixed $k = 4$: the throughput $S \lfloor k/2 \rfloor$ grows linearly, and T_5 falls until it hits the unlimited floor $2\chi'$, at $S = 3$ for C_{12} and already at $S = 2$ for K_8 , after which further switches sit idle.

Switch count can never buy *feasibility*, however. A single MPP measurement must connect its entire support through one switch, so K_5 ’s Case 4 remains exactly as infeasible with two four-port switches as with one; only port width $k \geq \Delta + 2$ admits it. Scaling out with many narrow switches therefore serves CZ-based preparation and conflict-free MPP parallelism, while dense-graph MPP is served only by scaling the port width up.

6.7 Verification and Cross-Validation

Every number in this chapter passed the runtime checks of Chapter 5, and three independent lines of evidence support their correctness. First, the theoretical bounds held

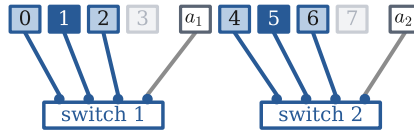


Figure 6.18: One Tock of Case 4 (MPP) for C_8 on two four-port switches, one measurement per switch.

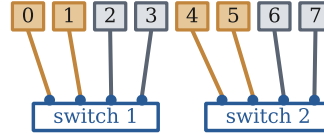


Figure 6.19: One round of Case 5 (CZ) for C_8 on two four-port switches, two gates per switch.

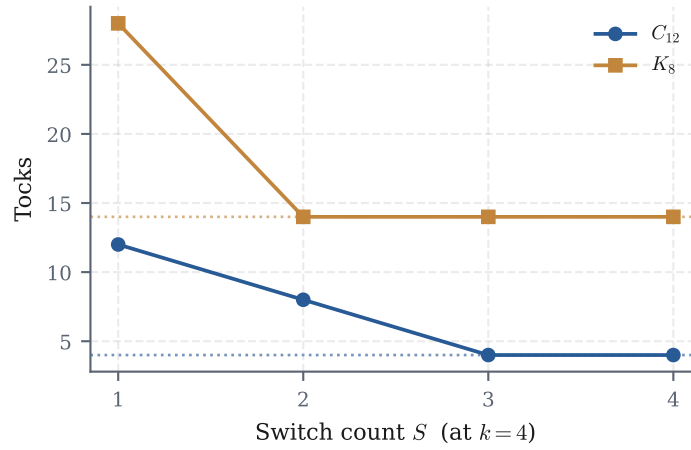


Figure 6.20: Measured Case-5 (CZ) depth versus switch count at fixed $k = 4$, with each graph's unlimited floor dotted.

on every instance: each Case-1 schedule satisfied $\omega(\mathcal{C}) \leq T_1$, each greedy colouring stayed within $\Delta \leq \chi' \leq 2\Delta - 1$, and the throughput inequalities $T_2 \leq T_3$ and $T_2 \leq T_5$, which must hold because capping how many gates run at once can never speed the CZ schedule up, were never violated. Cases 1 and 4 carry no such ordering: Case 4 trades the substrate's interval blocking for the switch cap, and Section 6.6.1 measures both directions.

Second, the Case-3 and Case-5 schedulers, implemented independently, are provably identical when $k_a = S \lfloor k/2 \rfloor$, and their outputs agreed on every row of every table in this chapter, including the survey, where the matched parameters make the T_3 and T_5 columns coincide exactly. Third, the measured hardware staircases reproduce the closed form $T_3 = 2 \sum_i \lceil |C_i|/k_a \rceil$ point for point. Where the heuristics fall short of

optimal, the simulator reports it: the complete-graph staircase in Figure 6.16 is flagged as greedy colouring overhead rather than physics, and the congestion excess $T_1 - \omega$ of Section 6.4.3 is reported alongside its conflict floor.

Finally, every experiment is seeded and scripted: the survey, scaling, crossover, and hardware sweeps regenerate bit-identically from a single command.

6.8 Discussion

The main conclusion of this chapter is not that one operation beats the other, but a clear picture of when each one wins. On depth, the two flips of Section 6.4 bound MPP’s territory from opposite sides. Measurement conflicts hand the incidence-style graphs to CZ outright, and routing congestion hands large two-dimensional graphs to CZ on any linear-bus architecture, while paths, cycles, stars, cliques, and small dense graphs remain MPP’s by a wide margin. On hardware, CZ gates consume two ports regardless of the graph, so they run unmodified on narrow switches where MPP is excluded outright, and the survey shows this exclusion is the common situation: on a four-port switch, four of the six families lose MPP entirely, and no number of such switches restores them, since a measurement cannot straddle two switches. The realistic design question is therefore three questions in sequence: where the target graph sits relative to the conflict boundary, whether the architecture’s routing keeps the congestion term small, and whether the switch is wide enough for MPP at all.

The results also locate where each resource matters. Sparse, low-degree graph states, including paths and cycles, are essentially free in time, costing a constant two to four Ticks at any size, so for them the binding constraint is the QPU count $n + k_a$, not the schedule. Star-like and dense states invert the picture: their CZ schedules grow linearly while MPP stays shallow, so wide switches ($k \geq \Delta + 2$) buy large constant factors, a factor of fourteen already at the eight-qubit star. The grid states underlying MBQC are the most consequential case: at small sizes MPP halves the schedule and needs six-port switches, but beyond the 5×5 grid the congestion of the one-dimensional substrate erodes the advantage entirely, so large cluster states on a linear bus should be prepared with CZ, and recovering MPP’s advantage at scale requires two-dimensional routing. These are concrete, checkable inputs to a photonic-switch specification.

Three caveats bound these conclusions, and they point in different directions. The Tock metric counts schedule depth only, and routing a many-qubit MPP is physically harder than a pairwise CZ, so the metric flatters MPP. The greedy edge colouring inflates the reported CZ depths, visible as the complete-graph staircase, so the CZ numbers are upper bounds that flatter MPP as well. The one-dimensional substrate

penalises MPP through congestion that richer routing would partly recover, flattering CZ. That the two operations trade wins even under biases pointing both ways is itself evidence that the crossover is real rather than an artefact of the model.

6.9 Chapter Summary

This chapter turned DQS-Sim’s measurements into a design map. The worked examples traced each schedule down to individual switch ports, showing how the vertex-to-qubit mapping makes an ancilla patch block every substrate position it spans, and how each measurement wires data and ancilla QPUs through the switch ports. The MPP-versus-CZ study found that neither operation dominates: MPP is shallowest on all six survey families, but CZ overtakes it through two separate mechanisms, conflict density on projective-plane incidence graphs, where the guarantee $\omega \geq 2\chi' \Rightarrow T_1 \geq T_2$ holds and the CZ advantage grows without bound, and layout congestion on square grids, where ω stays flat yet the one-dimensional substrate hands grids beyond 5×5 to CZ.

The scaling study showed that sparse graph states cost a size-independent constant number of Tocks while dense families grow linearly; the hardware study yielded explicit sizing thresholds, $k_a \geq \max_i |C_i|$ and $k \geq \Delta + 2$, at which each resource stops paying for itself, and separated switch count, which buys parallel throughput, from switch width, which alone buys MPP feasibility. All reported numbers passed runtime bound checks, agreed across independently implemented schedulers where theory demands agreement, and regenerate deterministically from scripted sweeps.

Chapter 7

Conclusion

7.1 Summary of Contributions

This thesis has made the following contributions to the study of distributed fault-tolerant quantum computing:

1. **DQS-Sim:** The first open-source simulator for estimating the fault-tolerant cost of graph state preparation in a distributed QPU architecture, covering five scheduling scenarios from fully idealised (MPP, unlimited ancillas) to fully constrained (CZ, switch-limited).
2. **Five scheduling algorithms:** Algorithms for MPP scheduling (Cases 1 and 4) and CZ scheduling (Cases 2, 3, and 5), with explicit handling of ancilla pool constraints and k -port switch connectivity.
3. **Theoretical bound verification:** Automatic runtime verification against the clique-number lower bound, the greedy colouring upper bound, and Vizing’s theorem, ensuring scheduling correctness.
4. **Physical resource formulae:** Closed-form expressions for total physical qubit counts as a function of graph structure, code distance, ancilla pool size, and switch parameters.
5. **Measured design guidance:** A survey across six graph families, a two-sided crossover study, a scaling study, and hardware sweeps of the ancilla pool and switch port count. These yield a measured rule for the operation choice: CZ is guaranteed faster when the conflict clique satisfies $\omega(\mathcal{C}) \geq 2\chi'(G)$, and overtakes MPP through routing congestion on large two-dimensional graphs even below that boundary, while MPP leads elsewhere. The hardware sweeps add a feasibility requirement of $\Delta + 2$ switch ports for MPP and explicit sizing thresholds ($k_a \geq \max_i |C_i|$, $k \geq \Delta + 2$).

7.2 Limitations and Future Work

Several limitations of the present work point to directions for improvement. The greedy edge colouring used for the CZ cases can use up to $2\Delta - 1$ colours where the Vizing optimum is Δ or $\Delta + 1$, so replacing it with an optimal bipartite or Vizing-optimal colouring would tighten the Case 2 depths and shift the crossover boundary of Chapter 6 slightly further in CZ’s favour. In the same vein, the maximum independent set is computed exactly only for $n \leq 20$ and approximated greedily beyond that, and a stronger approximation would sharpen the Case 1 Tock count for large structured graphs.

On the modelling side, DQS-Sim assumes a single-level switch topology: the S switches are mutually unconnected fabrics (Section 2.8.3), so an operation must fit within one switch, and switch count can never substitute for port width. Adding switch-to-switch trunk links would relax exactly that rule. Linking S switches of k ports with t trunk connections yields a virtual switch of roughly $Sk - 2t$ usable ports, so two trunked four-port switches would just admit the six-port measurements of K_5 that Chapter 6 reports as infeasible; the price is trunk ports and one additional photonic hop of insertion loss per crossing. Quantifying that trade, alongside a probabilistic photon-loss model with retries, would make the estimates more realistic, while integrating a magic-state distillation cost model would enable full algorithm-level resource estimation beyond the Clifford operations considered here.

The most significant direction, however, is the move from static to dynamic compilation. DQS-Sim assumes *static* compilation, in which the entire graph state is generated before any qubit is measured, so the hardware must hold a surface code patch for every vertex of the state at once. Measurement-based computation does not require this: a qubit can be measured as soon as its neighbours are in place, freeing its slot for reuse and sharply reducing the peak number of qubits needed at any one time, a quantity known as the spatial cost and governed by the graph’s pathwidth. Extending the simulator to schedule this *dynamic* generation and measurement under GoSC, and to quantify the qubit and Tock savings of dynamic over static compilation in the distributed setting, is the natural continuation of this work, a direction the distributed compilation literature likewise identifies as open [28], since static compilation becomes infeasible for the large graph states that practical algorithms demand.

7.3 Broader Impact

As quantum hardware scales towards fault-tolerant regimes, the ability to accurately estimate and optimise resource costs becomes critical for hardware design decisions. DQS-Sim provides a lightweight, graph-aware tool that complements heavier compilation-pipeline approaches (BenchQ, Azure Quantum Resource Estimator) by focusing specifically on the graph state preparation bottleneck. Its explicit modelling of the distributed QPU constraints, namely ancilla pools and switch connectivity, fills a gap in the existing toolchain and provides actionable guidance for quantum network architects.

Bibliography

- [1] Sitong Liu, Naphan Benchasattabuse, Darcy QC Morgan, Michal Hajdušek, Simon J Devitt, and Rodney Van Meter. A substrate scheduler for compiling arbitrary fault-tolerant graph states. In *2023 IEEE International Conference on Quantum Computing and Engineering (QCE)*, volume 1, pages 870–880. IEEE, 2023.
- [2] Thaddeus D Ladd, Fedor Jelezko, Raymond Laflamme, Yasunobu Nakamura, Christopher Monroe, and Jeremy L O’Brien. Quantum computers. *Nature*, 464(7285):45–53, 2010.
- [3] Peter W Shor. Algorithms for quantum computation: discrete logarithms and factoring. In *Proceedings 35th Annual Symposium on Foundations of Computer Science*, pages 124–134. IEEE, 1994.
- [4] Lov K Grover. A fast quantum mechanical algorithm for database search. In *Proceedings of the Twenty-Eighth Annual ACM Symposium on Theory of Computing*, pages 212–219. ACM, 1996.
- [5] John Preskill. Quantum computing in the nisq era and beyond. *Quantum*, 2:79, 2018.
- [6] Robert Raussendorf and Hans J Briegel. A one-way quantum computer. *Physical Review Letters*, 86(22):5188, 2001.
- [7] Marc Hein, Wolfgang Dür, Jens Eisert, Robert Raussendorf, Maarten Van den Nest, and Hans J Briegel. Entanglement in graph states and its applications. *arXiv preprint quant-ph/0602096*, 2006.
- [8] Hans J Briegel and Robert Raussendorf. Persistent entanglement in arrays of interacting particles. *Physical Review Letters*, 86(5):910, 2001.
- [9] Michael A Nielsen. Cluster-state quantum computation. *Reports on Mathematical Physics*, 57(1):147–161, 2006.
- [10] Daniel Gottesman. *Stabilizer codes and quantum error correction*. PhD thesis, California Institute of Technology, 1997. arXiv:quant-ph/9705052.

- [11] Simon J Devitt, William J Munro, and Kae Nemoto. Quantum error correction for beginners. *Reports on Progress in Physics*, 76(7):076001, 2013.
- [12] Barbara M Terhal. Quantum error correction for quantum memories. *Reviews of Modern Physics*, 87(2):307–346, 2015.
- [13] William K Wootters and Wojciech H Zurek. A single quantum cannot be cloned. *Nature*, 299:802–803, 1982.
- [14] Alexei Yu Kitaev. Fault-tolerant quantum computation by anyons. *Annals of Physics*, 303(1):2–30, 2003.
- [15] Sergey B Bravyi and Alexei Yu Kitaev. Quantum codes on a lattice with boundary. *arXiv preprint quant-ph/9811052*, 1998.
- [16] Eric Dennis, Alexei Kitaev, Andrew Landahl, and John Preskill. Topological quantum memory. *Journal of Mathematical Physics*, 43(9):4452–4505, 2002.
- [17] Austin G Fowler, Matteo Mariantoni, John M Martinis, and Andrew N Cleland. Surface codes: Towards practical large-scale quantum computation. *Physical Review A*, 86(3):032324, 2012.
- [18] Clare Horsman, Austin G Fowler, Simon Devitt, and Rodney Van Meter. Surface code quantum computing by lattice surgery. *New Journal of Physics*, 14(12):123011, 2012.
- [19] Daniel Litinski. A game of surface codes: Large-scale quantum computing with lattice surgery. *Quantum*, 3:128, 2019.
- [20] Lu-Ming Duan, Mikhail D Lukin, Juan Ignacio Cirac, and Peter Zoller. Long-distance quantum communication with atomic ensembles and linear optics. *Nature*, 414(6862):413–418, 2001.
- [21] William J Munro, Koji Azuma, Kiyoshi Tamaki, and Kae Nemoto. Inside quantum repeaters. *IEEE Journal of Selected Topics in Quantum Electronics*, 21(3):78–90, 2015.
- [22] Jens Eisert, Kurt Jacobs, Panagiotis Papadopoulos, and Martin B Plenio. Optimal local implementation of nonlocal quantum gates. *Physical Review A*, 62(5):052317, 2000.
- [23] J Ignacio Cirac, Artur K Ekert, Susana F Huelga, and Chiara Macchiavello. Distributed quantum computation over noisy channels. *Physical Review A*, 59(6):4249, 1999.

- [24] Rodney Van Meter. *Architecture of a quantum multicomputer optimized for Shor's factoring algorithm*. PhD thesis, Keio University, 2006. arXiv:quant-ph/0607065.
- [25] Stephanie Wehner, David Elkouss, and Ronald Hanson. Quantum internet: A vision for the road ahead. *Science*, 362(6412):eaam9288, 2018.
- [26] Adán Cabello, Lars Eirik Danielsen, Antonio J López-Tarrida, and José R Portillo. Optimal preparation of graph states. *Physical Review A*, 83(4):042314, 2011.
- [27] Madhav Krishnan Vijayan, Alexandru Paler, Jason Gavriel, Casey R Myers, Peter P Rohde, and Simon J Devitt. Compilation of algorithm-specific graph states for quantum circuits. *Quantum Science and Technology*, 2024. arXiv:2209.07345.
- [28] A. Micciche, N. Benchasattabuse, A. McGregor, M. Hajdušek, R. Van Meter, and S. Krastanov. Quantum hamlets: Distributed compilation of large algorithmic graph states. arXiv preprint arXiv:2603.06387, 2026.
- [29] Michael E Beverland, Prakash Murali, Matthias Troyer, Krysta M Svore, Torsten Hoeffler, Vadym Kliuchnikov, Guang Hao Low, Mathias Soeken, Aarthi Sundaram, and Alexander Vaschillo. Assessing requirements to scale to practical quantum advantage. *arXiv preprint arXiv:2211.07629*, 2022.
- [30] Zapata Computing. BenchQ: Resource estimation for fault-tolerant quantum computation. <https://github.com/zapatacomputing/benchq>, 2023. Open-source software developed under the DARPA Quantum Benchmarking program.
- [31] Aric A Hagberg, Daniel A Schult, and Pieter J Swart. Exploring network structure, dynamics, and function using NetworkX. In *Proceedings of the 7th Python in Science Conference (SciPy 2008)*, pages 11–15, 2008.
- [32] Vadim G Vizing. On an estimate of the chromatic class of a p-graph. *Diskret. Analiz.*, 3:25–30, 1964.
- [33] Douglas Brent West. *Introduction to Graph Theory*. Prentice Hall, 2nd edition, 2001.